Excitation-Aware On-Track System Identification at Full Scale

Ebrahim Abdelghfar Ebrahim[1], Hazem Abuelanin[2], Mohamed Abdelaziz[1], Mohamed Omar Mohamed Elsayed[1], Sherif Said[1]

***Abstract*—On-track system identification promises to learn tire models from ordinary racing laps instead of dedicated steady-state manoeuvres. We study the identifiability of one such residual-learning pipeline at full vehicle scale, in simulation, on a plant that publishes its own per-wheel slip angles, loads and lateral forces, so the estimator can be scored against the forces the plant actually generated. Transplanted unchanged from a small-scale platform, the pipeline converges on a tire with less than half the vehicle's peak grip and most Pacejka coefficients resting on their box constraints. This is not a solver, bounds or tuning failure but a loss of excitation created by the pipeline's own virtual-data step: the friction the synthetic rollout can demand, $\mu_{\mathrm{reach}} = v^2\delta_{\mathrm{sweep}}/(gL)$, is a property of the rollout configuration and not of the tire. We give that identifiability bound and verify it directly—on a grid over rollout speed and true peak factor with a known tire, every tire below $\mu_{\mathrm{reach}} \approx D$ comes back as the same wrong number, independently of solver and start point—and correct the pipeline by setting the rollout speed from the measured lap, bounding the steering sweep to the observed range, freezing the regularisation target, identifying against achieved rather than commanded steering, and reading a friction floor from the *curvature* of the axle force curve. Admissibility gates on derived physical quantities guard the model-to-controller boundary. Against the inherited configuration over three timed laps, no identified coefficient rests on a bound and the peak factor stays within 0.05 of the plant's effective friction, against 0.77. All results are in simulation and each configuration is one run.**

***Index Terms*—System identification, tire modeling, Pacejka Magic Formula, identifiability, persistent excitation, vehicle dynamics, autonomous racing, simulation architecture.**

I. Introduction

Model-based controllers for autonomous racing derive their performance from the fidelity of the vehicle model they carry [1], [2], [3]. In the lateral channel that fidelity is dominated by the tire, which is simultaneously the physical bound on manoeuvrability and the least accessible component to measure: its behaviour depends on surface, temperature, wear and load in ways that change between sessions [4], [5].

Classical characterisation resolves this by removing the vehicle from the track—steady-state circular or ramp-steer manoeuvres in a large open space yield clean data at a logistical cost a race weekend rarely affords [6]. On-track identification removes the logistics but reintroduces transients, noise and, critically, the fact that a racing line is a poor excitation signal for a nonlinear model; nonlinear least squares applied directly to lap data is known to be brittle in exactly this regime [6].

[1]The authors are with the Faculty of Engineering, Ain Shams University, Cairo, Egypt, and the Autotronics Research Lab (ARL), Cairo, Egypt.

Corresponding author: Ebrahim Abdelghfar Ebrahim (e-mail: ebrahimabdelghfar550@gmail.com).

Dikici *et al.* [6] propose the hybrid remedy that is the starting point of this paper. A small residual network is trained on 30 s of ordinary lap data to predict the one-step error of a nominal single-track model; the corrected model is then rolled out through a *synthetic*, quasi-steady steering sweep; and Pacejka coefficients are fitted to that synthetic sweep. Because the sweep is generated rather than driven it can be made steady and noise-free, and because the residual network only has to interpolate within its training distribution its generalisation weakness is contained. The procedure iterates, each fit becoming the next nominal model. On a 1:10 platform it reports a 3.3× lower one-step RMSE than nonlinear least squares under noise, from 30 s of driving [6].

A. Problem statement

This paper asks what the pipeline identifies when it is moved from a 3.7 kg, 0.32 m-wheelbase platform to a full-scale Formula Student vehicle: 269.8 kg, 1.53 m wheelbase, 16 ° of road-wheel lock, driven at 9.6–25.8 m/s. The question is one of identifiability rather than of parameter transfer. Two scale-dependent mechanisms, both latent at 1:10, break the estimate:

1) **Excitation collapses inside the virtual-data step.** The Pacejka fit never sees the recorded lap; it sees the synthetic rollout, whose reachable friction $\mu_{\mathrm{reach}} \approx v^2\delta_{\max}/(gL)$ is a property of the rollout configuration, not of the tire. A rollout speed inherited from the scaled platform caps it at 0.43 here. Below the tire's peak the Magic Formula degenerates to $F_y \approx F_z(BCD)\alpha$: only the product BCD is identifiable and the bounded optimiser resolves the remaining freedom on a box edge [7], [4].
2) **Lateral demand scales as v^2, so a bad grip estimate is silent until it is catastrophic.** Driven through the same controller on an earlier, faster reference trajectory, one identified set with a peak friction coefficient of 0.40 tracked at 0.16 m of cross-track error at 11.1 m/s, 0.43 m at 15 m/s and 41.89 m at 27 m/s. The scaled platform never reaches the speed at which the error becomes observable. The figures are from an offline closed-loop harness rather than from a controlled simulator run and are reported as such throughout (Section VII-A).

A third difference is an opportunity. Because the study is conducted in the CARLA simulator [8] through a purpose-built ROS 2 interface, the simulator's own per-wheel telemetry—slip angle, normal load, lateral force, per wheel—is available as ground truth, so an on-track identifier can be scored against the forces the plant actually generated.

B. Contributions

The contributions are an identifiability result, the pipeline changes it implies, and the simulation architecture that makes both verifiable.

1) **An identifiability analysis of the virtual-data step** (Section V-B), giving the closed-form reachable-friction bound above and an *excitation-aware rollout* that selects the rollout speed from the measured lap and warns whenever the configuration cannot reach the prior's peak. The bound is verified directly in Section VI-B, on a grid over rollout speed and true peak factor with a known tire: below $\mu_{\mathrm{reach}} \approx D$ the fit returns the excitation rather than the tire, at every grip level tested and independently of the solver. This corrects the baseline method [6] at any scale; it is latent at 1:10 because a 2–4 m/s rollout is representative there and is not here.
2) **Two mechanisms that stabilise the iterative co-identification** (Section V-C): the synthetic sweep is bounded at $1.5\times$ the 99th percentile of the observed steering, so the fit cannot read the residual network's extrapolation as grip; and the Tikhonov target is frozen at the configured prior while the optimiser's start point tracks the iterations. Together they remove a $1.01\rightarrow1.87$ drift of the peak factor over six iterations.
3) **Measurement-side corrections for full-scale deployment** (Section V-E): slip angles built from the *achieved* road-wheel angle rather than the command; mass and wheelbase from measured static wheel loads rather than from a configuration file; and a friction warm-start computed as a high quantile over the whole buffer and applied as a *floor* rather than as a median over a low-slip mask.
4) **A simulation architecture for on-track identification research** (Sections IV-B–IV-E). CARLA_ASU_BRIDGE exposes the plant's own per-wheel forces, slip angles and loads as ROS 2 topics alongside a conventional autonomy interface, and closes the identification loop end to end—acquisition, preprocessing, fitting, ground-truth force validation—on a single simulation clock.
5) **Admissibility gates on derived physical quantities** (Section VII) at the model-to-controller boundary—axle cornering stiffness, understeer sign and critical speed, and grip ceiling against the trajectory's lateral demand—rather than per-coefficient box checks, which cannot detect a degenerate fit or a pairwise-oversteering axle set.
6) **A closed-loop evaluation against the inherited configuration** (Section VI), on a static 1.05 friction surface over three timed laps, scored throughout against the simulator's own per-wheel forces. The corrected pipeline holds its peak factor within 0.05 of the plant's effective friction where the inherited configuration starts and ends on its box constraint, and halves the lateral tracking error under MPC. We report the two axes on which the inherited configuration is better, and the reason: the lap reaches 2.2° of slip, where a wrong peak factor costs nothing.

The paper follows that order: Section II places the work against the state of the art; Section III states the identification problem and its structural identifiability properties; Section IV describes the simulation architecture and data pipeline; Section V the identification methodology; Section VI the results; Section VII how the identified model is handed to the controller; and Sections VIII–IX the limitations and outlook. All results are obtained in simulation, and Section VIII states which conclusions are expected to transfer to hardware.

II. Related Work and Position

A. Tire models and Magic Formula identification

The Magic Formula of Bakker, Nyborg and Pacejka [9] and its consolidation [10], [4] remains the standard empirical description of steady-state tire force generation and is the model identified here; its lateral form and the two structural properties this paper relies on are stated in Section III. Classical characterisation obtains the coefficients from steady-state manoeuvres in a controlled space [6], [5]: clean data at high logistical cost. Direct fitting to measured contact-patch forces is the most informative variant and requires a force-measuring hub or a simulator's own wheel telemetry; in this work it is used only as a *reference* against which the on-track identifier is scored, never inside the identification loop. Between the two sits interactive identification from track or rig data, of which TRIP-ID [11] is representative: an operator-guided fit of the full Magic Formula to recorded manoeuvres. The pipeline studied here targets the same output with no operator and no dedicated manoeuvre, which is precisely what puts its excitation—and therefore its identifiability—outside the engineer's control.

B. On-track and data-driven identification

On-track methods trade excitation quality for operational simplicity. Nonlinear least squares on lap data is the classical baseline and is brittle under noise [6]. The recursive alternative to a batch fit is a dual or joint filter that carries states and parameters together—Wenzel *et al.* [12] is the canonical vehicle instance—and it is subject to the same excitation condition this paper is about: a filter run on a smooth racing line has as little information about the tire's peak as a batch fit of the same data, and expresses it as a parameter covariance that does not shrink rather than as a coefficient on a bound. Dedicated friction estimators make the excitation dependence explicit [13], [14], [15], and Hsu *et al.* [16] is the closest prior art to the warm start of Section V-E3: it recovers the friction limit below the peak from the *curvature* of the force response, read through the pneumatic trail in steering torque. Our estimator uses the same observation on a different signal—the axle force curve reconstructed from the IMU and odometry, with no steering-torque sensor and no assumption about the steering system—and is gated and applied as a one-way floor, which is a property of the interface to the controller rather than of the estimator. Gaussian-process approaches learn the mismatch of a nominal model and have been used both for identification and inside the controller [17], [18], [2]; they are accurate but carry a continuing computational cost and, as [6] observes, are usually evaluated starting from an already accurate physical model. Neural residuals replace the Gaussian process with a cheaper regressor at the cost of generalisation guarantees.

Deep Dynamics [19] is the closest work on the physics-constraint axis: it estimates Pacejka coefficients inside a neural network and adds a *Physics Guard* layer clamping the internal estimates into nominal ranges. Our admissibility gates share the motivation but differ in placement and in what is constrained. We report as a substantive finding that per-coefficient box constraints are *insufficient*: the degenerate fit that motivated this work satisfied every box constraint while five of eight coefficients sat exactly on a bound, and a later fit satisfied every box constraint while being pairwise over-steering and open-loop unstable above 14.5 m/s. We therefore gate on derived quantities at the model-to-controller interface and treat a coefficient resting on a bound as a diagnostic of non-identifiability rather than as a successful clamp.

The hybrid residual-plus-virtual-data structure of [6] is the pipeline we extend. Its key idea—query the learned model only where it interpolates, and extract physically parameterised coefficients from it—is retained unchanged. What we correct is the mechanism that decides *where* the learned model is queried.

C. Overlap with concurrent work

Wu *et al.* [20] extend the same baseline concurrently and independently, and the overlap must be stated plainly. They introduce (i) a vision-based friction prior from a lightweight CNN over road texture, used to warm-start the optimisation; (ii) an S4 structured state-space model in place of the MLP; and (iii) Nelder–Mead for the iterative Pacejka extraction, with virtual simulation in CarSim. The pipeline described here independently contains an S4D temporal residual [21], [22], a friction warm-start, and Nelder–Mead and differential-evolution solver options alongside trust-region reflective. We therefore do *not* claim the temporal-residual architecture, the friction warm-start concept, or the derivative-free solver as contributions.

Three differences are technical rather than chronological. First, our friction prior is proprioceptive—derived from measured accelerations, in the spirit of model-free friction estimation from inertial data [23]—and therefore needs no camera or textured-surface assumption. Second, a prior computed from *utilised* friction is a lower bound on *available* friction and cannot substitute for excitation: on a gentle lap the median utilisation on our vehicle measures 0.044. We therefore identify the peak from the curvature of the axle force curve, by a gated two-parameter brush-model fit [24] of the kind surveyed by Acosta *et al.* [15], and—whatever the fit's own confidence—apply the result only as a floor, for a reason that belongs to the interface with the controller rather than to the estimator (Section V-E3d). The actual fix for the degeneracy is located in the rollout excitation, not in the initialisation. Third, we report the cost of the temporal residual and reduce it by 8.7× without changing its output (Section V-A4), which is what makes an S4-family residual a deployable option on a vehicle CPU rather than an offline one; the techniques are standard for diagonal state-space models [21], [22], [25] and we claim only the measurement and the exactness. To our knowledge the reachable-friction bound of Section V-B and its consequence for the virtual-data step have not previously been stated.

D. Simulation architectures for identification and validation

CARLA [8] is the standard open simulator for autonomous-driving research and supplies the sensor and scenario infrastructure used here. Its native dynamics come from the underlying PhysX wheeled-vehicle model [26]—a saturating friction-and-stiffness tire law (Section IV-C) rather than an empirical Magic Formula fit to measured data—which is why this study treats the simulator as a well-instrumented *plant* to be identified and not as a source of ground-truth Pacejka coefficients. Recent work addresses that fidelity gap directly: R-CARLA [27] makes the dynamics model interchangeable while retaining CARLA's sensor simulation, and Sagmeister *et al.* [28] quantify how far a vehicle model can be simplified before conclusions about a trajectory-following controller stop transferring—finding, relevantly here, that sensitivity is largest near the acceleration limits. CARLA has also hosted complete Formula Student Driverless stacks with ROS 2 interfaces [29]. The architecture of Section IV sits in this line and is distinguished by exposing the simulator's own per-wheel slip, load and force telemetry on a simulation-time-stamped interface, so that an identification method can be scored against the forces the plant actually produced rather than against a secondary model. Scaled platforms—F1TENTH [30] and the ForzaETH Race Stack [3]—provide the software context of the baseline; full-scale Formula Student Driverless systems [5] the vehicle class targeted here.

E. Downstream consumer and gap addressed

The identified model is consumed by a nonlinear MPC path-tracking controller built on `acados` [31] with partial-condensing HPIPM [32], following standard practice in racing MPC [1], [2], tracking a minimum-curvature raceline generated in the manner of Heilmeier *et al.* [33]. We claim no contribution to the controller itself; the controller appears in this paper only as the consumer that defines what an admissible identification is (Section VII).

Summarising the gap: the baseline [6] establishes that a residual network plus virtual steady-state data recovers a Pacejka model from a lap at 1:10 scale, and concurrent work [20] improves its residual architecture and its initialisation. Neither addresses the condition under which the virtual data itself carries information about the tire's peak, which is the binding constraint at full scale, and neither treats the admissibility of the identified model for its downstream controller as an explicit, physically derived contract. This paper addresses both, on a platform where the answer can be checked against the plant's own forces.

III. Identification Problem

A. Model structure

The identification model is the dynamic single-track model of the baseline, with mass m, yaw inertia I_z and centre-of-gravity distances l_f, l_r, $L = l_f + l_r$. Longitudinal and lateral dynamics are decoupled and load transfer is neglected:

$$\dot{v}_y = \tfrac{1}{m}\big(F_{y,r} + F_{y,f}\cos\delta - m v_x \omega\big), \tag{1}$$
$$\dot{\omega} = \tfrac{1}{I_z}\big(F_{y,f} l_f \cos\delta - F_{y,r} l_r\big), \tag{2}$$

with slip angles

$$\alpha_f = \delta - \arctan\Big(\tfrac{v_y + l_f\omega}{v_x}\Big), \quad \alpha_r = -\arctan\Big(\tfrac{v_y - l_r\omega}{v_x}\Big). \tag{3}$$

Axle forces follow the lateral Magic Formula

$$F_y = F_z\, D \sin\big(C \arctan\big(B\alpha - E(B\alpha - \arctan B\alpha)\big)\big), \tag{4}$$

with per-axle coefficients $\Phi_p = [B_f, C_f, D_f, E_f, B_r, C_r, D_r, E_r]$. The identification state and input are $\mathbf{x} = [v_y, \omega]$ and $\mathbf{u} = [v_x, \delta]$, and the nominal one-step map is $\hat{\mathbf{x}}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k, \Phi_p)$. The baseline discretises (1)–(2) by explicit Euler at T_s; we retain Euler as the default and expose second- and fourth-order Runge–Kutta as options, because the full-scale vehicle's lateral mode is stiffer relative to T_s than the scaled platform's.

B. Objective and derived quantities

The identification objective is to recover Φ_p from a single 30 s segment of ordinary lap data, without dedicated manoeuvres, such that the resulting model is both predictive and *admissible* for the controller that consumes it. Admissibility is expressed on derived quantities rather than on coefficients. The linearised axle cornering stiffness is the slope of (4) at zero slip,

$$C_i = F_{z,i} B_i C_i^{\mathrm{MF}} D_i, \qquad i \in \{f, r\}, \tag{5}$$

(writing C^{MF} for the shape factor to avoid a clash), and the grip ceiling implied by an identified set is

$$a_y^{\max} = \frac{D_f F_{z,f} + D_r F_{z,r}}{m}. \tag{6}$$

The linear model built from (5) is oversteering when $l_f C_f > l_r C_r$ and is then open-loop unstable above the critical speed $v_{\mathrm{crit}} = L\sqrt{C_f C_r / \big(m(l_f C_f - l_r C_r)\big)}$ [34].

C. Structural identifiability

Two properties of (4) are used throughout. First, its slope at zero slip is $F_z BCD$: the cornering stiffness depends on the coefficients only through that product, and E drops out. Second, and decisively, if the data does not approach the peak then (4) linearises to $F_y \approx F_z(BCD)\alpha$ and B, C, D become mutually unidentifiable. This is a structural-identifiability statement in the sense of Bellman and Åström [35] combined with the standard persistent-excitation argument [7], and neither is claimed as novel. What we claim, and demonstrate in Section V-B, is that the baseline pipeline's virtual-data step can silently violate it, and that a bounded optimiser then resolves the remaining freedom by sliding onto a box edge—so the violation is reported as a converged fit rather than as a failure.

D. Baseline pipeline

The baseline proceeds in four stages, repeated for N_{it} iterations [6]:

1) **Residual learning.** A small network f_{NN} is trained on the recorded lap to predict the one-step error $e_k = \mathbf{x}_{k+1} - \hat{\mathbf{x}}_{k+1}$ from $[v_x, v_y, \omega, \delta]_k$; the corrected model is $\mathbf{x}_{k+1} = \hat{\mathbf{x}}_{k+1} + e_k$.
2) **Virtual steady-state generation.** The corrected model is integrated open-loop at constant longitudinal speed while the steering angle is ramped from 0 to δ_{sweep}, producing a synthetic quasi-steady dataset.
3) **Pacejka fit.** Assuming $\dot{v}_y = \dot{\omega} = 0$ on the synthetic data, axle forces are recovered from the yaw balance,
$$F_{y,r} = \frac{m l_f v_x \omega}{L}, \qquad F_{y,f} = \frac{m l_r v_x \omega}{L\cos\delta}, \tag{7}$$
slip angles from (3), and Φ_p is fitted to $(\alpha_i, F_{y,i})$ by bounded least squares.
4) **Re-nomination.** The fitted Φ_p becomes the next iteration's nominal model and the residual network is reinitialised.

The structural point that drives the rest of this paper is that stage 3 *never sees the recorded lap*. It sees the output of stage 2. The recorded lap enters only through the residual network, so the excitation available to the fit is set by the rollout configuration and not by how the vehicle was driven.

IV. Simulation Architecture and Data Pipeline

The identification pipeline is exercised end to end inside a single simulation architecture: a Formula Student vehicle model in CARLA acting as the plant, a C++ ROS 2 communication layer that drives it and republishes its state, and an identification stack that consumes that state. The architecture is described here because two of its properties—per-wheel ground truth and simulation-time stamping—are what make the identifiability claims of Section V checkable rather than merely arguable.

A. Choice of simulator

CARLA [8] was selected against four requirements that the identification problem of Section III imposes on a simulator.

Per-wheel ground truth. The claim to be checked is that the recovered Magic Formula reproduces the lateral force the plant actually produced, so the simulator must expose per-wheel slip angle, normal load and tire force—not only chassis states. CARLA's PhysX wheeled-vehicle solver computes these quantities internally and its client API exports them, which is what makes the tire forces feedback channel of Section IV-D possible.

Deterministic, simulation-time-stamped execution. Persistent-excitation and residual-learning arguments are only checkable if every sample carries a consistent clock. CARLA's synchronous mode with a fixed physics step (0.033 s here) decouples data timing from wall-clock load; all identification data is stamped from the simulation clock (Appendix, Section A7).

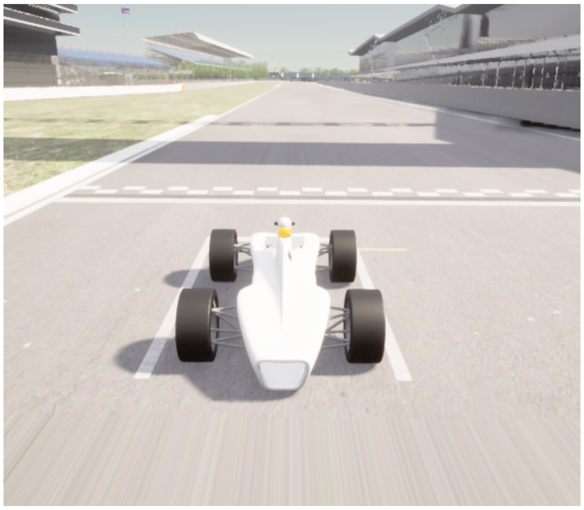


Fig. 1. The Formula AI vehicle model, on the start point of the silverstone map in CARLA. Its lateral forces are produced by the engine's PhysX wheeled-vehicle tire model [26] rather than by a Magic Formula, which is the curve of Table II the identifier is scored against. Measured geometry and mass are given in Table I.

Full-scale vehicle and racing-track support. The target class is a full-scale Formula Student car on a closed circuit. CARLA accepts custom vehicle assets—here the Formula AI vehicle on the `silverstone` map—and offers a client that the bridge links against directly. Surveys of open self-driving simulators place CARLA at the state of the art for end-to-end stack testing on exactly these grounds [36].

B. Plant: Formula AI vehicle in CARLA

The plant is the driverless Formula Student–class vehicle model, instantiated in CARLA [8] (server 0.9.16, Unreal Engine 4.26) on the `silverstone` map: a four-wheeled, front-steered, all-wheel-drive rigid-body vehicle whose lateral forces are generated by the PhysX wheeled-vehicle tire model [26] under a configured friction coefficient and lateral stiffness, not by a Magic Formula. The simulator therefore has no "true" Pacejka coefficients to recover; instead, the lateral forces PhysX itself computes at each wheel are taken as ground truth, and the identified Magic Formula is validated by comparing its predictions against those forces rather than against any coefficient the simulator holds (Section V-E).

C. The plant's tire curve, in closed form

The identification target is worth stating exactly, because it is knowable here. PhysX computes the lateral force of each wheel with `PxVehicleComputeTireForceDefault` [26]; at zero longitudinal slip and zero camber that expression collapses to

$$F_y(\alpha) = \mathrm{sgn}(\alpha)\,\mu F_z\,S_1(K), \qquad K = \frac{C_{\mathrm{lat}}|\tan\alpha|}{\mu F_z}, \tag{8}$$

with $C_{\mathrm{lat}} = F_z^{\mathrm{rest}}\,k_y\,S_1\big(3\bar{F}_z/k_x\big)$, $\bar{F}_z = F_z/F_z^{\mathrm{rest}}$ the normalised load, (k_y, k_x) the wheel's lateral-stiffness parameters, and $S_1(K) = \min(1,\, K - K^2/3 + K^3/27)$ the engine's smoothing function.

TABLE I
VEHICLE CONFIGURATION AND MEASURED PLANT PROPERTIES.

Quantity	Value	Unit
Body mass (configured)	240.0	kg
Static wheel load (total)	2644.5	N
Total (kerb) mass m[a]	269.8	kg
Unsprung mass[b]	29.8	kg
Front/rear static split	51.2 / 48.8	%
Wheelbase L	1.5334	m
l_f / l_r	0.7381 / 0.7953	m
Yaw inertia I_z[c]	51.1	kg m^2
Wheel radius	0.25	m
Max road-wheel angle	16.0	°
Configured wheel friction	1.5	—
Road-surface factor	0.70	—
Effective friction[d]	1.05	—
Realised peak axle friction	1.00–1.05	—
Drag coefficient	0.426	—

Configured values are set in the bridge's `carla_interface_config.yaml`; measured values are obtained from the simulator's per-wheel telemetry. [a]Total static wheel load divided by g. This is the mass every equation in this paper uses, not the configured body mass. [b]Total minus body mass; not measured independently. [c]Assumed; see Section A. [d]Configured wheel friction multiplied by the road surface's own coefficient, as reported per wheel by `feedback/tire_forces`. It is the effective value, not the configured one, that the identifier is scored against (Section V-E3d).

TABLE II
THE PLANT'S LATERAL TIRE CURVE: THE PARAMETERS OF (8), READ FROM THE SIMULATOR'S OWN WHEEL CONFIGURATION AND TELEMETRY.

Quantity	Value	Unit
Effective friction μ	1.05	—
Lateral stiffness k_y	17.0	/rad
Max normalised load k_x	2.0	—
Initial slope C_{lat} at $\bar{F}_z = 1$	14.875 F_z^{rest}	N/rad
front axle, $F_{z,f} = 1373\,\mathrm{N}$	20.4	kN/rad
rear axle, $F_{z,r} = 1274\,\mathrm{N}$	19.0	kN/rad
Peak reached at, $\bar{F}_z = 1$, $\mu = 1.05$	12.0	°

The initial slope is set by the *rest* load and by the normalised load through S_1, not by F_z alone: $C_{\mathrm{lat}} = k_y S_1(3\bar{F}_z/k_x)\,F_z^{\mathrm{rest}}$ evaluates to $14.875\,F_z^{\mathrm{rest}}$ at $\bar{F}_z = 1$ and falls monotonically with $\bar{F}_z$.

Every axle slope quoted in this paper is therefore taken at $\bar{F}_z = 1$ and at the static axle loads listed here (the geometric split l_r/L, l_f/L of the total wheel load; see Section A, A2, for its 1.4 % disagreement with the measured load split of Table I).

The peak angle follows from $K = 3$ in (8), i.e. $\tan\alpha_{\mathrm{peak}} = 3\mu/\big(k_y S_1(3\bar{F}_z/k_x)\big)$, and therefore moves with μ: 12.0 ° at the effective $\mu = 1.05$.

Two properties of (8) shape everything downstream. It saturates at μF_z, first reached at $K = 3$, and it *never falls off* past that peak—so a Magic Formula, whose E and post-peak decay exist precisely to describe a falling branch, can match it up to the peak and not beyond. And its scale is set entirely by μ and C_{lat}, both of which the simulator publishes, so the curve of Table II is read off the plant rather than fitted to it.

That the closed form is the right one was checked rather than assumed. On 1804 ground-truth wheel samples from a live lap ($P_{99}(|\alpha|) = 3.66\,°$), each predicted from its own measured α, F_z and μ, (8) returns $R^2 = 0.99$ with 16.1 N RMSE on the front axle and 14.7 N on the rear, at biases of +2.0 N and +0.8 N. Section VI-D uses this curve to separate the error of the identified fit from the error of representing (8) in the form

TABLE III
ROS 2 INTERFACE USED BY THE IDENTIFICATION AND VALIDATION PIPELINE. NAMESPACE IS /sim UNLESS MARKED ABSOLUTE.

Topic	Type	Role
/odom	nav_msgs/Odometry	State v_x, v_y, ω; twist is body-frame
feedback/steering_angle	std_msgs/Float32 (deg)	Achieved road-wheel angle δ
feedback/imu	sensor_msgs/Imu	a_x, a_y for the friction warm-start
feedback/tire_forces	sim_manager_msgs/TireForces	Per-wheel α, F_z, F_y, μ, ω_w, κ: **ground truth**
feedback/vehicle_physics	std_msgs/String (JSON)	Live wheel parameters of (8)
feedback/speed	std_msgs/Float32	Forward speed echo
feedback/motors	std_msgs/String (JSON)	Per-wheel speed/torque/brake
/drive	ackermann_msgs/AckermannDriveStamped	Control command in
control/tire_friction	std_msgs/Float32	Runtime grip override
/clock *(abs.)*	rosgraph_msgs/Clock	Simulation time base

(4) that the controller requires.

D. CARLA_ASU_BRIDGE*: interfaces and telemetry*

CARLA_ASU_BRIDGE [37] is a C++ ROS 2 lifecycle node linked directly against CARLA's client library (Fig. 2). It spawns and configures the ego vehicle, drives its inputs, and republishes simulator state as standard ROS 2 topics; all identification traffic crosses this interface.

1) Command path: The bridge runs in `ackermann_drive` mode, subscribing to a single `ackermann_msgs/AckermannDriveStamped` on `/drive` and forwarding speed and steering angle to CARLA's native Ackermann controller, whose server-side cascaded PID loops are configured once at startup. No client-side steering PID is present—the commanded road-wheel angle passes through in radians—so the achieved/commanded ratio of Table I is a property of the vehicle and its steering curve rather than of a tuning choice in the bridge.

2) Feedback path: Table III lists the interface consumed by the identifier and its validation. One property of it is specific to this architecture.

a) Per-wheel ground truth: `feedback/tire_forces` carries CARLA's own `WheelTelemetryData`: per-wheel lateral slip angle, normal load, lateral force, effective friction coefficient, wheel speed, longitudinal force and wheel torque, in wheel order [FL, FR, RL, RR]. No analytic tire model exists inside the bridge, so nothing on this topic can disagree with the physics the vehicle is driven by. This topic is what allows an on-track identifier to be scored against the forces the plant generated. Four properties of it were established by measurement and bound what may be read off it. (i) Only `lateral_force` is force ground truth; it is the channel every validation in this paper uses. (ii) `longitudinal_force` is a friction-capacity report rather than a measurement—it tracks μF_z and correlates with ma_x at 0.04—and `wheel_torque` is an exact identity of it, so neither carries independent information. (iii) Below 0.5 m/s the publisher zeroes slip angle and both forces while the normal load stays live, so a load-only gate admits standstill frames as perfect $(0, 0)$ samples; the identifier gates on the all-zero pattern as well. (iv) The slip angle needs no sign change across the boundary, which was verified rather than assumed.

Figure 3 states the conventions the rest of the paper assumes. The simulator and the identification stack do not share a handedness: CARLA inherits Unreal's left-handed world (X east, Y *south*, Z up, yaw positive clockwise), while every ROS 2 topic follows REP-103 [38]—right-handed ENU for the world, forward-left-up for the body—and the identification equations of Section V are written in the body frame with α and F_y positive to the left, the SAE-style axis convention [39], [4]. The bridge is where the two meet, and it is a per-channel conversion rather than a single transform: position and yaw flip sign, twist is additionally rotated into the body frame by q^{-1}, the IMU's a_y flips, the steering *command* flips on the way out, and the tire telemetry's slip angle does *not*, because the frame flip and CARLA's own sign convention for `lat_slip` cancel. That last one is the trap: a sign error there is invisible in the states and inverts the identified curve, so it was checked against a slip angle rebuilt from the chassis state in a steady 4 m/s turn at 0.20 rad—equal magnitude to within 2 % front and 15 % rear.

3) Extensions made for identification: Six changes support this study and are listed for reproducibility: the existing but unsubscribed `feedback/steering_angle` is now consumed by the identifier (with configurable units, scale and staleness timeout, falling back to the `/drive` setpoint); runtime overrides `control/tire_friction` and `control/drag_coefficient` write straight to the live `VehiclePhysicsControl`, so surface friction can change within a run without respawning; the torque curve, zero-throttle damping and wheel `tire_friction` were tuned against a low-speed hold test (a wheel friction above ≈2.0 on a steered wheel scrubs hard enough at creep speed to stall the drivetrain intermittently), and these values define the plant being identified; telemetry runs on a dedicated 10 Hz thread off the control loop, since each tick costs several blocking CARLA RPCs; `feedback/tire_forces` was extended with the effective friction coefficient, the wheel speed and a kinematically recomputed slip ratio $\kappa = (\omega_w r_w - v)/v$, the last because CARLA's own `long_slip` field is inconsistent with the wheel speed and radius the same message reports (+0.177, i.e. hard braking, on a wheel turning 8.99 rad/s at a steady 2.33 m/s cruise, whose kinematic ratio is -0.034); and the live wheel parameters of (8) are published on `feedback/vehicle_physics`, so the reference curve of Table II cannot go stale when the vehicle configuration changes.

E. Data acquisition and preprocessing

The vehicle is driven around the reference raceline by a model-free pure-pursuit controller, which requires no tire knowledge—only the wheelbase—and is therefore a valid bootstrap, as in the baseline [6], [3]. Odometry, the achieved steering angle and the drive command are sampled into a ring buffer by a 50 Hz timer for 30 s, gated on $v_x > 1$ m/s.

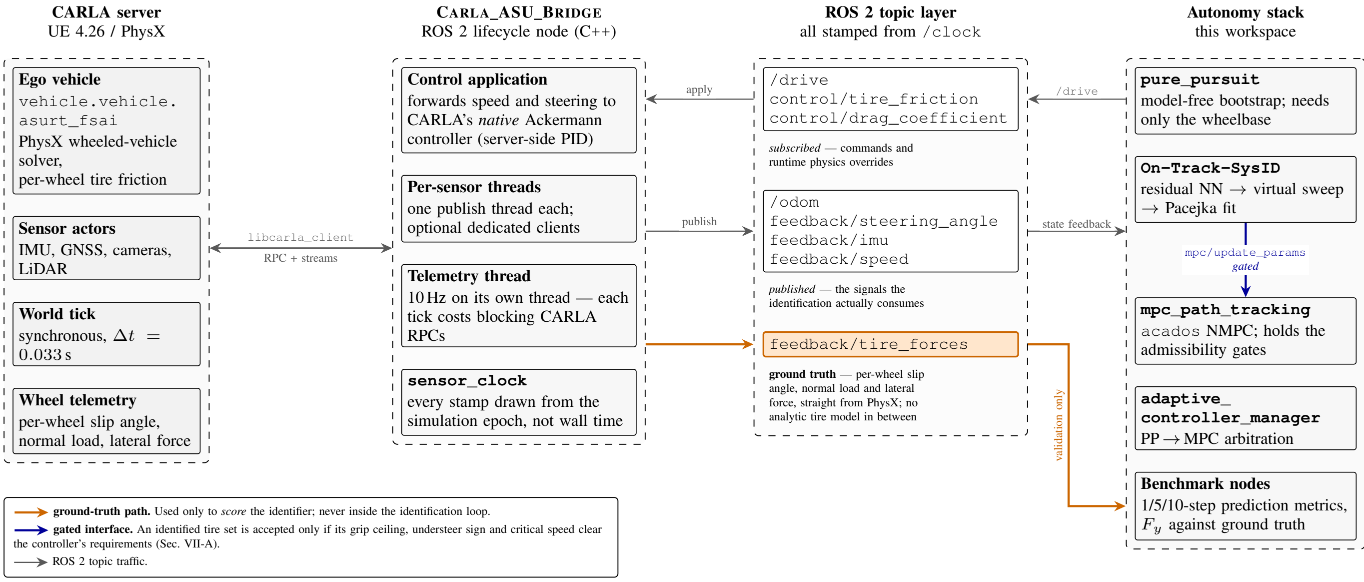
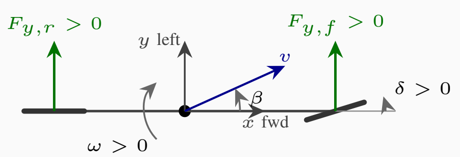


Fig. 2. Architecture of the CARLA_ASU_BRIDGE ROS 2 interface. The bridge links against CARLA's C++ client, drives the ego vehicle through CARLA's native Ackermann controller, and republishes state, sensors and per-wheel telemetry on ROS 2 topics stamped from the simulation clock. The `feedback/tire_forces` path (highlighted) carries the simulator's own per-wheel slip angle, normal load and lateral force, and is the ground truth against which the identifier is scored.

(a) Vehicle body frame (FLU)



$$\alpha_f = \delta - \arctan\frac{v_y + l_f\omega}{v_x}, \quad \alpha_r = -\arctan\frac{v_y - l_r\omega}{v_x}$$

Positive α produces positive (leftward) F_y. Wheel order in every per-wheel array: [FL, FR, RL, RR].

(b) Conversions applied by CARLA_ASU_BRIDGE

Channel	Bridge conversion
`/odom` pose	$y \mapsto -y$; $\psi, \theta \mapsto -\psi, -\theta$
`/odom` twist	world $\rightarrow$ body by q^{-1}; v_y, ω_y, ω_z negated
`feedback/imu`	$a_y \mapsto -a_y$
`tire_forces` α	deg $\rightarrow$ rad only: the frame flip and CARLA's own sign convention cancel
`tire_forces` F_y	$F_y \mapsto -F_y$, into $+y$ left
`/drive` δ	negated on the way out to CARLA

Fig. 3. Frame and sign conventions across the simulator–ROS 2 boundary. (a) All identification quantities are defined in the body frame: x forward, y left, ω and δ positive counter-clockwise, and slip angle and lateral force positive to the left, so that a positive α produces a positive F_y [39]. (b) The conversion applied per channel. The slip angle of the tire telemetry is the one exception that needs no sign change, because the frame flip and CARLA's own convention cancel.

The timer runs on the simulation clock, which advances only at the server's 0.033 s step, so the buffer cannot acquire a new state faster than the physics produces one and the achieved acquisition rate is the step rate, not the timer rate. The interval the one-step model of Section III-A is discretised at is therefore set separately, at $T_s = 0.035$ s, to match the physics step rather than the timer. The residual 6 % mismatch between the two is not free of consequence in principle—a timer faster than the step returns repeated states, and the one-step residual target $e_k = \mathbf{x}_{k+1} - \hat{\mathbf{x}}_{k+1}$ then carries a zero-order-hold artefact—so the identification was repeated at both settings. The same two configurations identified at $T_s = 0.020$ s and at $T_s = 0.035$ s return the *same* last accepted coefficient set to four decimal places in the corrected configuration ($B, C, D, E = 7.076, 1.346, 1.009, -2.000$ front and $7.873, 1.383, 1.002, -1.024$ rear), and in the inherited configuration the coefficients move (B_f 4.72→4.80, E_f -2.44→-2.26) while D stays on its 2.0 bound on both axles at both settings. The identified tire is therefore insensitive to the mismatch, and the box-edge signature is not an artefact of it. Results in Section VI are reported at $T_s = 0.035$ s. Preprocessing follows the baseline: a zero-phase low-pass filter, then symmetry augmentation by mirroring $(v_y, \omega, \delta) \rightarrow (-v_y, -\omega, -\delta)$ at unchanged v_x, which doubles the data and removes the cornering bias of a circuit. Re-identification retriggers every 30 s by default.

F. How the architecture closes the loop

The four stages of the identification pipeline map onto the architecture as follows. *Acquisition* uses `/odom`, `feedback/steering_angle` and `/drive`, all stamped on the simulation clock, so that the interval the one-step model is discretised at can be matched to the interval the physics actually advanced (Section IV-E) rather than to the acquisition timer. *Preprocessing* and *fitting* run inside the identification node against that buffer. *Validation* draws on `feedback/tire_forces` and `feedback/imu`, which never enter the identification loop, so the estimator is scored against a channel it did not observe. *Deployment* pushes the fitted coefficients over a ROS 2 service to the controller behind the gates of Section VII. Only the last stage leaves the architecture, which is what lets Section VI attribute a closed-loop difference to the identified model rather than to the interface that carries it.

V. Identification Methodology

This section describes the identification pipeline. Section V-A covers the residual model, V-B–V-C the two changes to the virtual-data step that constitute the paper's central methodological contribution, V-D the fit itself, V-E the measurement chain, and V-G the validation protocol. Figure 4 summarises the loop.

A. Residual model

The residual carries whatever the nominal model (1)–(4) does not represent, and how that map is parameterised is a modelling decision rather than a hyperparameter. Two decisions are made here, both of the same kind: known vehicle physics is used to shape the residual's *input space* (Section V-A1) and, optionally, its *objective* (Section V-A2), so that the network's small capacity is spent on what is genuinely unmodelled instead of on rediscovering structure that (3) and (1)–(2) already state exactly. The rest of this subsection states both and why they were chosen; both are active in the configuration evaluated in Section VI, and neither is scored on its own.

The default residual is the baseline multilayer perceptron: 4 inputs, one hidden layer of 8 units, 2 outputs, LeakyReLU, 58 parameters, full-batch Adam [40] at 5×10^{-4} on an MSE objective, in PyTorch [41]. Because the full-scale residual has different structure from the scaled platform's, the architecture is selectable: `baseline`, `wide` (16 hidden units, 114 parameters, weight decay), `physics_inputs` (74 parameters), `ensemble` (three averaged members, 174 parameters), and `s4` (102–110 parameters). The results below use the baseline MSE objective, so that the identifiability statements of Sections V-B–V-C are not confounded with a change of residual.

1) Physics-augmented inputs: the slip angles: The baseline residual reads $\mathbf{z} = [v_x, v_y, \omega, \delta]$ and predicts $[e_{v_y}, e_\omega]$. The quantity it corrects, however, reaches the plant only through the slip angles of (3): the tire law (4) is a function of (α, F_z) and of nothing else in $\mathbf{z}$, so the target e is, to the extent the single-track structure holds at all, a function of $(\alpha_f, \alpha_r, F_{z,f}, F_{z,r})$ plus the genuinely unmodelled terms. The network is therefore asked to reconstruct α_f, α_r from $\mathbf{z}$ before it can represent anything.

That reconstruction is the worst part of the map to leave to a small network. Equation (3) composes a division by v_x with an arctangent; a one-hidden-layer piecewise-linear network approximates a *ratio* only by tiling the (v_y, ω, v_x) domain, and this vehicle's identification data spans 9.2–26.5 m/s, a $2.9\times$ range of the denominator. Eight LeakyReLU units cannot tile that, so the fitted residual represents the same tire nonlinearity independently at each speed it happened to observe, with nothing tying those representations together. The consequence is specific to this pipeline: stage 2 evaluates the network along a synthetic rollout at a single speed and a steering ramp, i.e. exactly in the region where such a residual has no reason to be consistent, and stage 3 then reads the result as tire force.

Appending α_f, α_r, computed from (3) with the *measured* geometry of Table I, hands the network the two arguments of (4) directly. Three properties motivated the choice. (i) The speed dependence becomes explicit rather than learned: a residual expressed in α transfers between the recorded lap's speed and the rollout speed by construction, which matters because Section V-B deliberately drives those two apart. (ii) It costs 16 weights ($58 \to 74$ parameters) and no new sensor, against the several hundred a network would need to tile the division; augmentation is the cheaper way to buy the same function. (iii) It is compatible with the symmetry argument of Section V-A2: α_f, α_r are odd in (v_y, ω, δ), so the mirrored batch mirrors the augmented inputs too and the symmetry penalty remains exact.

The raw four inputs are retained rather than replaced, because the effects the residual exists to absorb—load transfer, actuator dynamics, aerodynamic load, combined slip—are not functions of α alone, and v_x in particular carries the load-dependent part. The geometry used for the augmentation is the same l_f, l_r the fit and the force recovery use, which is why $l_{wb} \equiv l_f + l_r$ is enforced (Section V-E): an augmentation computed from a different wheelbase than the one (7) assumes would inject a bias exactly where the network is meant to remove one.

This is the physics-informed idea of [42], [19] applied at the input rather than at the loss, and the placement is deliberate: an input augmentation carries no penalty weight to tune, cannot trade against the data term, and costs nothing at inference, whereas a loss term buys the same structure only softly and only where the data supports it. Deep Dynamics [19] makes the complementary choice of constraining coefficients *inside* the network; here the network is left unconstrained and its input space is made physical instead. The augmentation is available on its own (`physics_inputs`) and combined with the temporal residual (**s4** with `use_physics_inputs`), and is rank-agnostic, so the same code path serves a flat batch and a windowed sequence batch.

2) Physics-informed objective: The optional physics-informed objective [42], [19] adds three penalties to the one-step MSE,

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_s \mathcal{L}_{\text{steady}} + \lambda_m \mathcal{L}_{\text{sym}} + \lambda_g \mathcal{L}_{\text{smooth}}, \qquad (9)$$

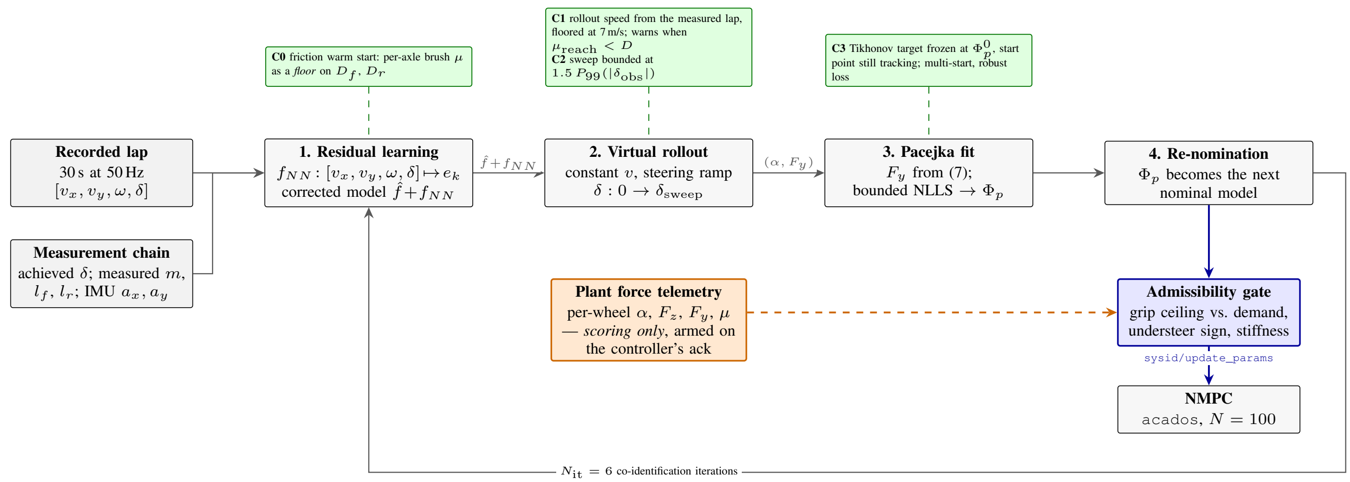


Fig. 4. The identification pipeline. Recorded lap data trains the residual network; the corrected model is rolled out through a synthetic steering sweep; Pacejka coefficients are fitted to that sweep; the fit becomes the next nominal model. Green marks this paper's changes to the inherited loop: the friction warm start applied as a floor on D (Section V-E3), the excitation-aware rollout speed and the extrapolation-bounded sweep (Sections V-B–V-C), and the frozen regularisation target of (12). Blue is the gated model interface to the controller (Section VII); orange is the plant's own per-wheel force telemetry, which scores the result, is armed only once the controller has acknowledged the submitted model (Section V-F), and is never admitted into the loop.

with $\lambda_s = 0.1$, $\lambda_m = 0.05$, $\lambda_g = 10^{-4}$. Each answers a specific weakness of training a residual on one lap of ordinary driving.

a) Dynamics consistency: $\mathcal{L}_{\text{steady}}$ substitutes the *corrected* next state $\mathbf{x}_{k+1} = \hat{\mathbf{x}}_{k+1} + e_k$ back into (1)–(2), with the axle forces re-evaluated through (4) at the corrected slip angles, and penalises the squared lateral and yaw balance residuals. Its purpose is to keep the two residual channels consistent with each other: nothing in the data term prevents an e_{v_y} and an e_ω that individually reduce one-step error while jointly implying an axle force pair no tire could produce, and it is that force pair, not the state prediction, that stage 3 subsequently fits. The term is applied only on a quasi-steady mask ($|\dot{v}_y| < 0.8\,\text{m/s}^2$, $|\dot{\omega}| < 6\,\text{rad/s}^2$), because (1)–(2) neglect load transfer and transient tire relaxation and therefore hold as written only where those terms are small; penalising the balance during a transient would ask the network to absorb the model's known omissions rather than the plant's unmodelled dynamics.

b) Left/right symmetry: The vehicle is laterally symmetric, so the true residual map is odd: $e(-v_y, -\omega, -\delta) = -e(v_y, \omega, \delta)$. $\mathcal{L}_{\text{sym}}$ penalises $\|f_{NN}(\mathbf{z}) + f_{NN}(-\mathbf{z})\|^2$ over the sign-mirrored batch. A circuit is rarely balanced in its corner directions, so 30 s of lap data covers one sign of δ far better than the other, and the synthetic sweep of stage 2 is run in one direction only. The mirror supplies the missing half of the input domain from geometry rather than from data, at no data cost; the same mirroring is applied to the recorded trace before windowing (Section V-A3), so the two uses are consistent.

c) Input-gradient smoothness: $\mathcal{L}_{\text{smooth}}$ penalises $\|\partial e/\partial \mathbf{z}\|^2$, a soft Lipschitz bound on the residual. Stage 2 queries the network beyond the steering range it was trained on, and Section V-C shows what an unconstrained extrapolation there costs. The two mechanisms are complementary and both are kept: the extrapolation bound of Section V-C decides *where* the sweep stops asking, and the gradient penalty decides *how fast* the residual is allowed to grow inside that range.

The nominal-model half of (9)—the nominal rollout, the mirrored inputs and the constant terms of the symmetry residual—depends only on quantities fixed for the whole training call and is precomputed once (Section V-A4).

3) Temporal residual: The `s4` architecture is an S4D diagonal state-space residual [21] over a sliding window of $W = 20$ samples (0.4 s, matched to tire-relaxation timescales). Windows are cut so that none straddles the seam between the recorded trace and its mirrored copy. Each block carries $H = 4$ independent single-input single-output channels of complex state dimension $N = 4$, initialised in the S4D-Lin form ($\text{Re}(A_n) = -\frac{1}{2}$, $\text{Im}(A_n) = \pi n$) that approximates the HiPPO-LegS operator [43] in closed form, with a learnable per-channel discretisation step. As stated in Section II, the S4-family residual overlaps with concurrent work [20] and is not claimed as a contribution; Section V-A4 reports what we did change, which is its cost. Figure 5 shows stage 1 in this configuration.

4) Evaluating the temporal residual at CPU cost: A temporal residual is only usable on-vehicle if it fits inside the re-identification period on the hardware the car carries, which is typically a CPU. Profiled as first written, the S4D path cost 125.7 ms per epoch on a GPU and 898 ms per stage-2 rollout, putting a six-iteration identification at 181.4 s—longer than the 30 s data window it consumes. Three changes remove that cost without altering the arithmetic. None is novel as a technique; what matters here is that the resulting pipeline is real-time-plausible on commodity hardware, and that the equivalence was verified rather than assumed.

a) Closed-form kernel instead of a scan: A diagonal A makes the block's impulse response available in closed form,

$$K_l = \text{Re}\big(C\, e^{A\Delta t\, l}\, \bar{B}\big), \qquad l = 0, \dots, W-1, \tag{10}$$

so a length-W window is one Vandermonde product followed by a causal Toeplitz matrix product, rather than W sequential

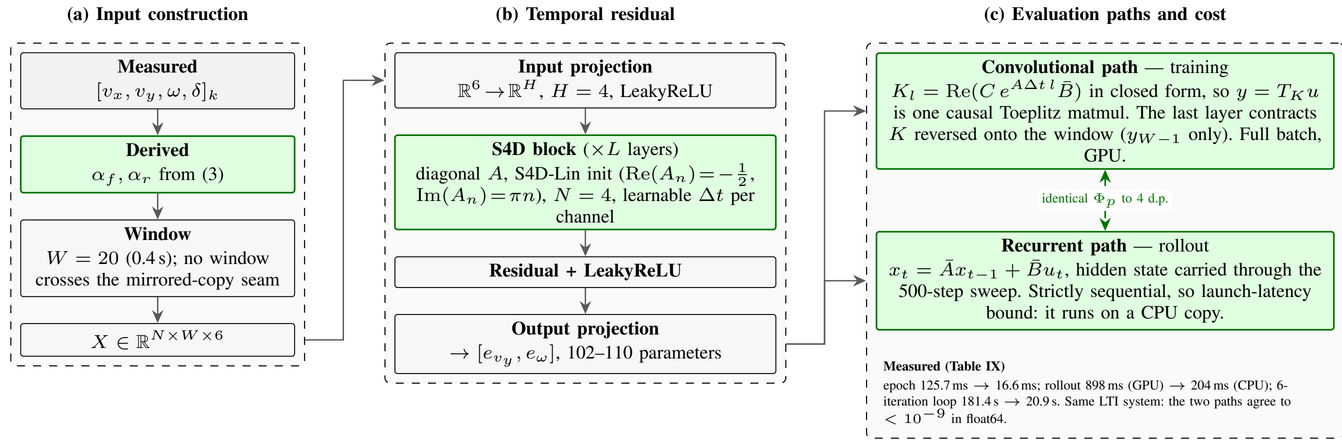


Fig. 5. Stage 1 of Fig. 4 with the physics-augmented input vector (Section V-A1) and the S4D temporal residual (Section V-A3). Green marks what differs from the baseline MLP residual. (a) The four measured signals are augmented with the two slip angles of (3) and cut into $W = 20$ sample windows. (b) The residual stack: input projection to H channels, S4D blocks with a diagonal state-transition matrix, and an output projection to $[e_{v_y}, e_\omega]$. (c) The same linear time-invariant system is evaluated two ways—as a causal convolution with the closed-form impulse response for training, and as a recurrence for stage 2's stateful rollout—which is what makes the architecture affordable on a CPU without changing a single identified coefficient (Section V-A4).

steps [21]. At $W = 20$ the dense Toeplitz product is cheaper than the FFT convolution of the original S4 [22] and than a parallel associative scan [25]; both are the right choice at sequence lengths orders of magnitude longer than this one. Because the stack's output layer reads only the window's last timestep, that layer contracts the reversed kernel straight onto the window and skips $W - 1$ of its outputs. Cost: 125.7 ms to 26.7 ms per epoch.

b) Loop-invariant physics-loss terms: Under the physics-informed objective, the nominal Pacejka rollout, the mirrored batch and the constant terms of the symmetry residual depend on the nominal model only, which is fixed for the whole training call, so they are precomputed once per call instead of once per epoch. The three forward passes the objective used to make (data, mirror, smoothness) collapse into one pass over a double-length batch, with the smoothness Jacobian taken off the same graph. Cost: 26.7 ms to 16.6 ms per epoch, with the loss value unchanged bit for bit.

c) The rollout belongs on a CPU: Stage 2 is 500 strictly sequential single-sample steps carrying the S4D hidden state, each with a host synchronisation. That is pure kernel-launch latency on a GPU: 898 ms on CUDA against 204 ms on a CPU copy of the same weights. The rollout is therefore run on a CPU copy whatever device training used—the recurrent path of Fig. 5(c)—and only the full-batch training stays on the GPU where it is 2.4× faster than a CPU. This is the one place where the two evaluation paths of the same system have to agree exactly, and they are pinned by a regression test to $< 10^{-9}$ in double precision.

End to end this takes a six-iteration identification from 181.4 s to 20.9 s (8.7×) on the same synthetic 30 s lap, with the identified Pacejka coefficients agreeing to four decimal places (Table IX). We deliberately did *not* touch the knobs that trade accuracy for speed—epoch count, iteration count, early stopping, warm-starting the network from the previous iteration's weights, or dropping the smoothness term—because those change the identified tire and are a modelling decision, not a free speedup.

B. Excitation-aware virtual rollout

This is the central correction. Consider stage 2 of Section III-D: a constant-speed rollout at v with the steering ramped to δ_{sweep}. In quasi-steady cornering the lateral acceleration a single-track vehicle develops kinematically is $a_y = v^2 \kappa$ with $\kappa \approx \delta / L$, so the largest friction coefficient the rollout can *demand* of the tire is

$$\boxed{\mu_{\text{reach}} \approx \frac{v^2\, \delta_{\text{sweep}}}{g\, L}} \tag{11}$$

Equation (11) contains no tire parameter: it is a property of the rollout configuration alone. If $\mu_{\text{reach}} < D$, the synthetic data never leaves the linear ramp of (4), only the product BCD is identifiable (Section III-C), and the bounded optimiser resolves the residual freedom on a box edge—the signature measured in Section VI-D.

The baseline implementation selects the rollout speed as $v = \mathrm{clip}(\overline{v_x}, 2, 4)$ m/s, a value representative of a 1:10 platform. On our vehicle, with $\delta_{\text{sweep}} = 0.4$ rad and $L = 1.5334$ m, (11) gives $\mu_{\text{reach}} = 0.43$ at 4 m/s: the rollout cannot ask the tire for more than $0.43\, g$ whatever the tire is. At 1:10 scale, with $L \approx 0.32$ m, the same rollout speed yields $\mu_{\text{reach}} \approx 2.0$ and the pathology is invisible.

1) Correction: The rollout speed is now taken from the speed the vehicle was actually driven at, clamped by configuration bounds (lower bound 15 m/s, no upper bound), and the pipeline warns whenever (11) evaluates below the prior's D—turning a silent degeneracy into a startup diagnostic. The same bound is the right tool for choosing a data-collection speed: identification data should be gathered at $v \geq \sqrt{gLD/\delta_{\text{sweep}}}$.

C. Bounded extrapolation and frozen-prior regularisation

Raising the rollout speed exposes a second, opposite failure. The rollout queries the residual network at steering angles it has never seen: on a pure-pursuit lap the observed $|\delta|$ stays below ≈ 0.06 rad while the sweep ran to 0.4 rad. The fit then reads the network's extrapolation as additional grip, and because each iteration's answer became both the next iteration's nominal model *and* its regularisation target, the error compounds: D walked from the 1.009/1.002 prior to 1.5402/1.8721 over six iterations, a claimed 1.7 g on a vehicle measuring 1.0 g in steady state.

Two changes close the loop:

1) **Extrapolation bound.** The sweep terminates at $\delta_{\text{sweep}} = \text{clip}\big(\lambda_e \cdot P_{99}(|\delta_{\text{obs}}|),\, \delta_{\min},\, \delta_{\max}\big)$ with $\lambda_e = 1.5$, $\delta_{\min} = 0.05\,\text{rad}$, $\delta_{\max} = 0.34\,\text{rad}$, so the network is never queried more than 50 % beyond the steering range it was trained on. Both $\delta_{\max}$ and the inherited $\delta_{\text{sweep}} = 0.4\,\text{rad}$ lie above the plant's 16.0° (0.279 rad) mechanical lock. That is deliberate: the sweep is a *virtual* rollout of the corrected model, and clamping it to the lock would cap μ_{reach} in (11) for a second time—by the actuator instead of by the rollout speed—which is the failure this section exists to remove. The binding limit in practice is the data-driven $\lambda_e P_{99}(|\delta_{\text{obs}}|)$ term, not $\delta_{\max}$; the residual is nonetheless evaluated outside the plant's reachable steering set, which is the extrapolation recorded in Section A, A6.
2) **Frozen prior.** The Tikhonov target is frozen at the configured prior Φ_p^0 for all iterations, while the optimiser's *start point* continues to track the previous iteration. The per-axle objective is

$$\min_{\phi} \sum_k \rho\big(r_k(\phi)\big) \;+\; \sum_j w_j\big(\phi_j - \phi_j^0\big)^2, \qquad (12)$$

with r_k the force residual, ρ a soft-ℓ_1 robust loss and w_j scaled to the data residual budget by a single dimensionless weight $\lambda = 0.05$. The regulariser exists specifically to hold the coefficients the manoeuvre does not excite—E above all—at their nominal value instead of on a box edge.

D. Pacejka fit

Fitting applies the steady-state force recovery of (7) to the synthetic data under box constraints $\Phi_p \in ([4.0, 1.2, 0.4, -3.0], [20.0, 2.2, 2.0, 1.0])$ per axle. Relative to the baseline we (i) raised the number of jittered multi-starts from 1 to 8, (ii) added a derivative-free simplex [44] and a global differential-evolution solver [45] as alternatives to trust-region reflective [46], all through the same optimisation library [47]. The prior carries the measured tire of Table II.

Neither (i) nor (ii) addresses the degeneracy, and both are verified as secondary in Section VI-B, which sweeps rollout speed against true D on a known tire: with the rollout speed left at 4 m/s every tire tested fits back as $\hat{D} \approx 0.5$ with coefficients railed, and the answer does not move with the multi-start seed or with a jitter seven times wider.

E. Measurement-side corrections

Three corrections apply to the measurement chain rather than to the estimator.

1) Achieved versus commanded steering: α_f in (3) was built from the `/drive` setpoint. The measured achieved/commanded ratio is 0.76, so α_f was systematically $\approx 25\,\%$ too large and the fitted front stiffness $\approx 20\,\%$ too small (13725 N/rad against 17198 N/rad fitted to the simulator's own per-wheel forces over the same run). Using `feedback/steering_angle` recovers 18486 N/rad. All three are effective slopes fitted over that run's load range and are therefore below the $\bar{F}_z = 1$ closed-form value of Table II; the comparison is between them and not against the tabulated slope.

2) Mass and geometry: m and l_{wb} are taken from the measured static wheel loads (Table I) rather than from the configuration file, and $l_{wb} \equiv l_f + l_r$ is enforced by a regression test, because (7) uses the wheelbase for the normal loads and the axle split for the force division; the two disagreeing is a silent bias.

3) Proprioceptive friction warm-start: The third correction is to where the peak factor's prior comes from. The inherited pipeline had none: D started from a configuration constant and was left to the fit. What follows recovers a prior for it from the sensors the vehicle already carries—axle forces and slip angles from the body's own motion (a), a two-parameter brush fit that reads μ from the *curvature* of those forces rather than from their asymptote (b), the release gate that says when that reading means anything (c), and the rule that the released value may only ever raise D (d).

a) Axle forces and slip angles without a force sensor: Newton–Euler on the single-track body of (1)–(2), solved for the axle forces rather than for the accelerations, gives

$$F_{y,f} = \frac{I_z\dot{\omega} + l_r m a_y}{L\cos\delta}, \qquad F_{y,r} = \frac{l_f m a_y - I_z\dot{\omega}}{L}, \qquad (13)$$

which requires the IMU's lateral acceleration and the yaw acceleration and no tire model at all. Vertical loads carry the longitudinal transfer $F_{z,f/r} = m(g\, l_{r/f} \mp a_x h_{cg})/L$ [34]. The yaw acceleration is taken as the derivative of a local polynomial fit to the yaw rate (Savitzky–Golay, window 21, order 2) [48] rather than by differencing, because $I_z\dot{\omega}$ is a $\approx 10\,\%$ term in $F_{y,f}$ on this vehicle and a differenced odometry yaw rate puts its noise straight into it. Slip angles come from (3) on the buffer that is already collected. Where no IMU is available the accelerations are differentiated from the states by the same filter, and where one is available the RMS difference between the two is logged as a bias diagnostic.

b) Joint fit of stiffness and peak friction: Per axle, both the cornering stiffness and the peak friction are fitted to the Fiala brush model [24], [49], [4]

$$F_y = \mu F_z\big(1 - (1-\xi)^3\big)\operatorname{sgn}\alpha, \quad \xi = \min\Big(\frac{C_\alpha|\tan\alpha|}{3\mu F_z}, 1\Big), \qquad (14)$$

by bounded nonlinear least squares with a soft-ℓ_1 loss [47], [46]. The brush model is used here in place of (4) for one reason: it has two parameters, both of which the manoeuvre can support, whereas (4) has four and is exactly the model whose non-identifiability at low slip this paper is about.

What makes this beat the utilisation bound is that μ enters (14) through the *curvature* of $F_y(\alpha)$ and not only through its asymptote. Writing $r = |F_y|/(C_\alpha|\tan\alpha|)$ for the force's departure from its own linear extrapolation, (14) inverts in closed form to

$$\mu = \frac{2k}{3 - \sqrt{3(4r-1)}}, \qquad k = \frac{C_\alpha|\tan\alpha|}{3F_z}, \tag{15}$$

so a 30 % departure from linearity ($r = 0.7$) already pins μ, well below the limit. Equation (15) is used only for the excitation diagnostics; C_α and μ are fitted jointly, because a two-stage fit would inherit the bias of estimating C_α on data that is itself already slightly nonlinear.

c) The gate is the feature: Equation (15) also states its own failure mode: $\mathrm{d}\mu/\mathrm{d}r$ grows without bound as $r \to 1$, i.e. as the tire curve stays linear. The estimate is therefore released only when three conditions hold—peak grip utilisation $\max|F_y|/(\hat{\mu}F_z) \geq 0.40$, relative precision $\sigma_{\hat{\mu}}/\hat{\mu} \leq 0.15$ from the Gauss–Newton covariance, and $\hat{\mu}$ not resting on its own search bound—and otherwise the module logs which condition failed and falls back to the utilisation bound.

The μ in the utilisation test is the *fitted* $\hat{\mu}$ of the same least-squares call, not an independent one, so that test is on its own circular: a fit that underestimates μ inflates its own utilisation and passes itself. It is therefore never used alone. The two companion conditions are what break the circularity—a fit driven low by data that is still linear returns a wide $\sigma_{\hat{\mu}}$, and a fit driven onto its search bound is rejected as railed—and the floor of Section V-E3d makes whatever still slips through one-sided, since an underestimate cannot lower D. The 40 % floor is not tuned here: it is the excitation level the lateral-dynamics friction-estimation literature converges on, as surveyed by Acosta *et al.* [15], whose review reports estimators failing below $|a_y| = 0.4\,g$ "due to the proximity of the tyre force curves in the low slip regions" and a 40–80 % utilisation working range. Samples with $|a_x| > 0.35\,g$ are dropped rather than corrected, since the axle longitudinal force split a friction-ellipse correction would need is not observable from this sensor set; samples below 2.5 m/s are dropped because (3) is ill-conditioned there; and samples whose force opposes their own slip angle are dropped as sign errors or noise.

d) Applied as a floor, unconditionally: The per-axle μ replaces the scalar bound as the initial guess for D_f and D_r where it is released, but—for either input, and however confident the fit—it can only ever *raise* D, never lower it. This is not conservatism for its own sake; two mechanisms depend on it. First, the frozen Tikhonov target of (12) is taken from the warm-started prior immediately after the warm start is applied, so whatever lands there is both the initial guess and the regularisation target of all six iterations. Second, the controller re-ingests the reference speed profile at $\min(a^{\mathrm{cfg}}_{y,\max},\, a^{\max}_y\eta)$ on every accepted model update, so a lowered D rewrites the reference under the running controller; a lowered D that also *moves* between cycles rewrites it repeatedly. The floor makes the estimate a one-way influence on the reference, which is the property the interface needs.

The gate cannot substitute for the floor, and it is worth being precise about why: σ_μ/μ from the fit covariance is a *precision* figure, not an accuracy one. A wrong mass, a wrong yaw inertia, an IMU bias or a road bank all fit tightly and wrongly through (13), since axle force scales linearly with m. On this vehicle the configuration file's 240.0 kg against the 269.8 kg the wheels actually carry biased μ 11 % low on its own—the 29.8 kg shortfall taken on the 269.8 kg the wheels carry—until the measured value replaced it (Table I); that is measurement-chain work (Section V-E) and not something a confidence interval can see. The same holds for the friction the result is compared against: the configured 1.5 is not the 1.05 the physics step uses, and an estimate validated against the former would be judged 30 % short $(0.45/1.5)$ while being correct. Of the inputs to (13), only I_z now remains unmeasured on this vehicle; it enters $F_{y,f}$ at the 10 % level and is listed among the assumptions of Section A.

F. Deployment

The identifier runs as a single ROS 2 node at 20 Hz with a rolling FIFO buffer of the collected signals, cycling through four states: collect a full buffer, train, await the controller's acknowledgement of the submitted coefficients, then run—re-identifying every $T_{\mathrm{reid}} = 30\,\mathrm{s}$ from the newest buffer. Three interface details matter for reproducing the results. The achieved road-wheel angle is used when the platform publishes one and falls back to the commanded setpoint on staleness, with the substitution logged (Section V-E). The friction warm start is recomputed on *every* cycle rather than once at startup, which is safe only because the static prior it floors is re-read from configuration each cycle and therefore cannot compound across cycles. Finally, the identified coefficients are submitted over a service call and the node keeps the previous model if the call is not acknowledged, so a rejected identification degrades to the last accepted model rather than to no model; the peak-friction outputs are published on a latched diagnostic topic so a late-starting consumer still sees the current value.

That acknowledgement is also what arms the scoring. The node's own online metrics, and the parameter forward to the passive benchmark node, both fire from the acknowledgement handler rather than from the end of training. The distinction is not cosmetic: a submission the controller *rejects* on its admissibility gate (Section VII) is a model the car never drove on, and the seconds between a fit completing and its acknowledgement landing are seconds in which the car is still driving the previous model. Scoring either of them attributes to the identified model behaviour that belongs to another one, which is exactly the confusion the validation protocol below exists to avoid.

G. Validation protocol

Three levels of validation are used, in increasing strength:

1) **One-step prediction** on a held-out run, as in the baseline: RMSE and R^2 of v_y and ω under $\hat{\mathbf{x}}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k, \Phi_p)$. A dedicated ROS 2 node extends this to configurable multi-step horizons (1/5/10 steps) with Euler/Heun/RK4 integration and online statistics in Welford's form [50]. Samples are admitted only above 1 m/s, the same gate the identification data itself uses. This is not a division-by-zero guard: since $\alpha = \arctan(v_y/v_x)$, a few tenths of a metre per second

of lateral drift saturates the tire curve and the one-step model predicts yaw rates of several rad/s against a ground truth of a few tenths, so a looser gate lets the low-speed bootstrap dominate a metric that is meant to describe the racing regime the model was fitted on.

2) **Direct force comparison** against `feedback/tire_forces`: the identified curve is evaluated at the measured (α, F_z) and compared per axle against the measured F_y, and against the plant's own curve (8) evaluated at the same points (Table II), which separates the error of the fit from the error of representing a PhysX tire.

3) **Closed-loop tracking** under the MPC using the identified model: mean and maximum lateral deviation, its time-weighted integral, heading error and solver cost, split by the active controller so that the pure-pursuit acquisition phase is not scored against a model it does not use. Lap counting runs from the first crossing of the raceline's $s = 0$, so the drive from the spawn point up to the start line is an untimed out-lap (Section VI-H).

All three levels score the *accepted* model only, for the reason given in Section V-F.

VI. Experiments and Results

A. Setup

All experiments run on the architecture of Section IV: the Formula AI vehicle in CARLA [8] driven through ROS 2 Humble, with the server in synchronous mode at a 0.033 s fixed physics step. The identifier collects 30 s of data at 50 Hz and re-identifies from the newest buffer, matching the baseline's data budget [6]. Ground truth for forces, slip angles and normal loads is the simulator's own per-wheel telemetry, which the identifier never reads.

1) Surface: Every run reported here uses a *static* friction surface: the per-wheel coefficient is held at its configured value of 1.5 for the whole run, and the road-surface factor of 0.70 that PhysX applies on top of it makes the effective coefficient the identifier is scored against 1.05, constant to the last digit the simulator reports over all 8186 and 9355 force-validation samples of the two runs. Time-varying friction is out of scope here (Section IX).

2) Reference trajectory: Both runs follow the same optimised raceline (Fig. 6), produced with the TUM global race trajectory optimisation package [51] under its minimum-curvature formulation [33] on a recorded centreline of the CARLA silverstone circuit: a closed 5828.2 m lap of 2916 waypoints at 2.0 m spacing, reference speed 9.62–25.78 m/s, minimum corner radius 18.7 m ($|\kappa|_{\max} = 0.0536$ /m). Its peak lateral demand is $\max(v_x^2|\kappa|) = 4.98\,\text{m/s}^2$ $(0.51\,g)$, which saturates the flat $5.0\,\text{m/s}^2$ `ggv` envelope the optimiser was given and stays at roughly half the $1.0\,g$ the plant sustains. This trajectory replaces the earlier one, whose $1.22\,g$ demand the admissibility gate of Section VII rejected against the same plant; the regeneration, not a relaxed gate, is what unblocked the closed-loop runs below.

TABLE IV
The two configurations.

Setting	`Baseline-NN-MSE`	`Ours`
Residual architecture	MLP (`baseline`)	S4D (`s4`)
Residual objective	One-step MSE	Physics-informed, (9)
Pacejka solver	Trust-region [46]	Differential evolution [45]
Friction warm start	Off	On, floor only

Everything not listed is identical between the two, including the excitation-aware rollout of Section V-B, which both use.

3) The two configurations: Two configurations of the same pipeline are run back to back on that trajectory, three timed laps each, with an untimed out-lap from the spawn point to the raceline's $s = 0$. They differ in the four settings of Table IV and in nothing else: the vehicle, the map, the trajectory, the controller, the data budget, the co-identification loop count and the Pacejka box constraints are shared. `Baseline-NN-MSE` is the inherited configuration—MLP residual, one-step MSE objective, local trust-region fit, no friction warm start—evaluated at full scale on the corrected rollout, so the comparison isolates the four settings rather than reproducing the published pipeline in full (that comparison is Table VII). `Ours` is the shipped configuration. Because the four settings move together, the comparison below is a configuration comparison and not a per-factor ablation. It is also not a test of this paper's central contribution: the excitation-aware rollout of Section V-B is active in *both* arms, so what Section VI-H measures is the four secondary settings on top of that correction. The correction itself is evidenced in Section VI-B.

4) What is measured: Three families of quantity are recorded, each against the plant's own telemetry: the identified tire model (coefficients, axle force, peak friction), the one-step state prediction, and the closed-loop tracking of the MPC that consumes the model. Every figure in this section is exported together with the CSV holding the samples behind it, and the cross-run comparison reads those CSVs rather than re-deriving any metric, so a figure and the table cannot disagree.

B. Identifiability of the peak factor against rollout excitation

The identifiability claim of Section V-B is a claim about the rollout configuration alone, so it is tested where the answer is known exactly rather than in CARLA. A tire with a known peak factor D is rolled out through the synthetic steering sweep of stage 2—the shipped rollout, with the residual network set to zero—and fitted back with the shipped solver. The fit is run *unregularised* and started from the true coefficients, so nothing the fit loses is lost to the prior, to the start point or to a local minimum; everything it loses is lost to excitation. Table V and Fig. 7 report the grid.

Two axes are swept, rollout speed v and true D, and each cell is repeated over five noise realisations. The repetition is over noise and not over the solver because the noise-free fit is exactly seed-invariant here: five multi-start seeds return the same coefficients to four decimal places, and so does a jitter width seven times wider, which is the direct measurement behind the statement that the degeneracy is not a

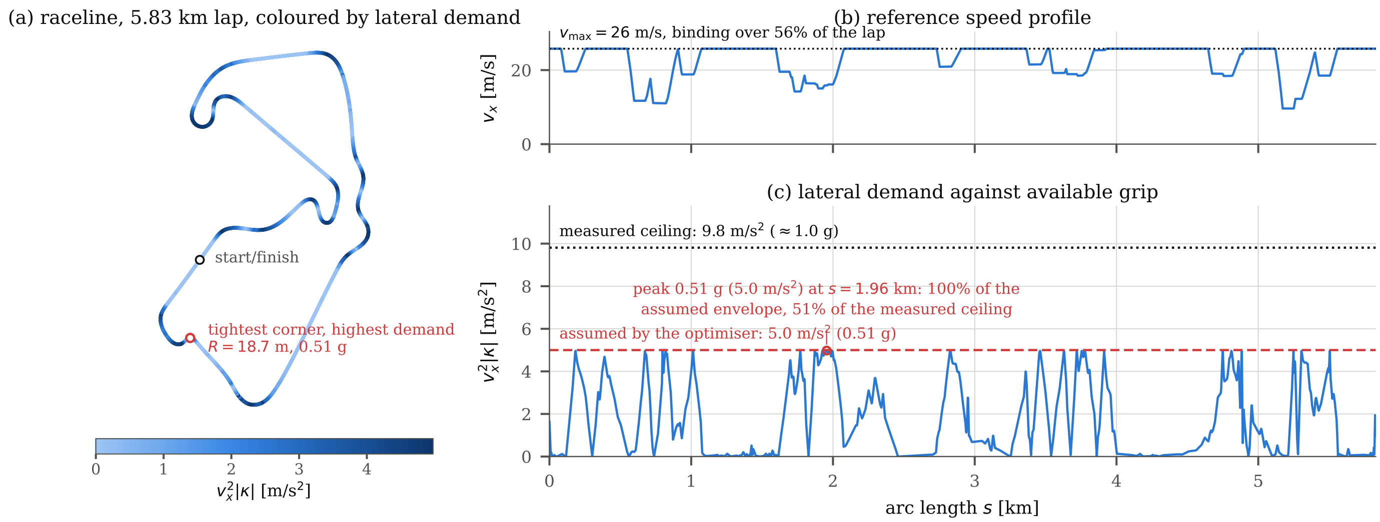


Fig. 6. The optimised raceline used for both runs, generated with the TUM global race trajectory optimisation package [51] under its minimum-curvature formulation [33] from a recorded centreline of the CARLA silverstone circuit. (a) Track layout, coloured by lateral demand; the marked corner ($R = 18.7$ m) is the tightest on the lap. (b) Reference speed profile, 9.62–25.78 m/s. (c) Lateral demand $v_x^2|\kappa|$ along the lap against the flat 5.0 m/s^2 envelope handed to the optimiser (dashed) and the grip the vehicle has (dotted, $\approx 1.0\,g$, measured). The profile saturates the envelope and no segment exceeds the plant's grip.

TABLE V
PEAK FACTOR $\hat{D}$ RECOVERED FROM A KNOWN TIRE, AGAINST ROLLOUT SPEED.

v	μ_{reach}	$\hat{D}$ (railed of 8) at true D of			
[m/s]		0.70	1.05	1.40	1.80
4	0.43	0.52±0.02 (1.8)	0.57±0.02 (2.8)	0.59±0.02 (3.8)	0.50±0.07 (7)
6	0.96	0.68±0.01 (1)	1.04±0.02 (1.2)	1.22±0.04 (1.8)	1.26±0.06 (2)
8	1.70	0.69±0.02 (1)	1.02±0.02 (0.8)	1.36±0.02 (0)	1.79±0.03 (1.2)
10	2.66	0.70±0.03 (0)	1.03±0.03 (0.4)	1.37±0.03 (1)	1.75±0.03 (0)
13	4.49	0.72±0.04 (0)	1.05±0.04 (0)	1.38±0.05 (0.4)	1.77±0.05 (0.4)
15	5.98	0.76±0.05 (1)	1.05±0.05 (0)	1.39±0.06 (0)	1.78±0.06 (0)
20	10.64	0.92±0.06 (1)	1.14±0.08 (0.4)	1.43±0.06 (0)	1.80±0.07 (0)

Unregularised, started from the true coefficients. Each cell is $\hat{D}$ (mean over ten fits, five noise realisations $\times$ two axles) $\pm$ the half-width of its 95 % interval, with the mean number of the eight coefficients resting on a box constraint in parentheses.
μ_{reach} is (11) at $\delta_{\text{sweep}} = 0.4$ rad and $L = 1.5334$ m. The inherited rollout speed is the first row.

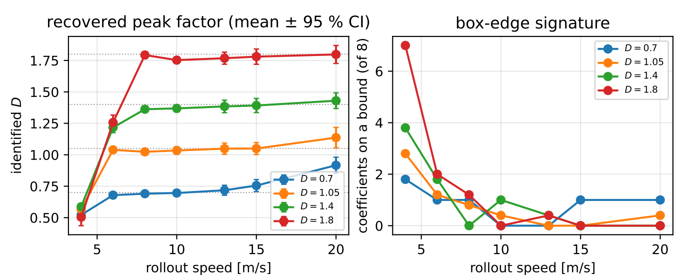


Fig. 7. The same grid as Table V. Left: recovered peak factor against rollout speed, one series per true D (dotted lines), mean $\pm$ 95 % interval over the noise realisations. Right: the mean number of coefficients resting on a box constraint. Below $\mu_{\text{reach}} \approx D$ the fit returns the excitation rather than the tire—at the inherited 4 m/s all four tires come back as $\hat{D} \approx 0.5$—and the coefficients slide onto the box. The transition is set by μ_{reach}, not by speed: at 6 m/s ($\mu_{\text{reach}} = 0.96$) the two low-grip tires are recovered and the two high-grip ones are not.

solver problem. The injected noise is Gaussian on the rolled-out states at the measured one-step persistence RMSE of the two CARLA runs, $\sigma_{v_y} = 0.009$ m/s and $\sigma_\omega = 0.005$ rad/s (Table VI), and is what the intervals in Table V are computed over.

The grid says three things. First, the inherited 4 m/s rollout is uninformative about D at *every* grip level tested: a 0.70, a 1.05, a 1.40 and a 1.80 tire all come back as $\hat{D}$ between 0.50 and 0.59, i.e. the fit reports μ_{reach} and not the tire, with two to seven of the eight coefficients on a bound. Second, the boundary is $\mu_{\text{reach}} \gtrsim D$ and not a speed: at 6 m/s, where $\mu_{\text{reach}} = 0.96$, the 0.70 and 1.05 tires are recovered to within 3 % while the 1.40 and 1.80 tires are 13 % and 30 % short. Above $\mu_{\text{reach}} = 1.7$ every tire is recovered to within 4 % with no systematic railing. Third, the failure is not the solver's: the noise-free fit is invariant to the multi-start seed and to the jitter width, so multi-start, a global solver or a better start point cannot recover a coefficient the data does not contain—which is why Section V-D reports the extra starts and the global solver as secondary. The one caveat is at the top of the sweep, where a rollout far past the peak begins to over-report a low-grip tire ($\hat{D} = 0.92$ for a 0.70 tire at 20 m/s), because the Magic Formula's falling branch is being fitted to data the plant's own curve does not produce; the extrapolation bound of Section V-C is what keeps the shipped configuration away from that regime.

C. Cross-run summary

Table VI is the complete comparison. `Ours` is better on peak friction, on rear-axle force, on yaw rate, on lateral tracking error in every phase, on both error tails and on the worst-case QP solve time; `Baseline-NN-MSE` is better on front-axle force, on lateral velocity and on the RMS QP solve time. The rest of this section reads those rows against the figures.

D. Identified tire model

Figure 8 puts both identified curves against the plant's own curve (8) and lists the coefficients. Two things are visible.

TABLE VI
CROSS-RUN COMPARISON OF THE TWO CONFIGURATIONS.

Metric	`Ours`	`Baseline-NN-MSE`
Closed-loop tracking (3 timed laps)		
Timed laps	3	3
Run duration [s]	591.4	564.1
Best lap [s], wall clock	148.60	142.56
Mean lap [s], wall clock	156.85	148.33
RMS e_y, overall [m]	**0.279**	0.303
RMS e_y, pure pursuit [m]	**0.405**	0.406
RMS e_y, MPC [m]	**0.092**	0.177
Max $\lvert e_y \rvert$ [m]	**2.651**	2.660
ITAE e_y [m s^2]	**10362**	17651
IAE e_y [m s]	**50.4**	74.2
IAE/T [m]	**0.085**	0.132
ITAE/$(T^2/2)$ [m]	**0.059**	0.111
RMS heading error [rad]	**0.0195**	0.0197
Max |heading error| [rad]	**0.311**	0.317
Emergency halts	0	0
MPC solve time, RMS [ms]	0.413	**0.390**
MPC solve time, max [ms]	**3.814**	4.061
Axle lateral force against per-wheel telemetry		
Front RMSE [N]	70.68	**45.90**
Rear RMSE [N]	**40.79**	50.06
Front R^2	0.944	**0.978**
Rear R^2	**0.979**	0.970
Front max abs. error [N]	**138**	215
Rear max abs. error [N]	**85**	223
Peak friction against the plant's $\mu = 1.05$		
Front μ RMSE	**0.041**	0.762
Rear μ RMSE	**0.048**	0.772
One-step state prediction		
v_y RMSE [m/s]	0.027	**0.022**
ω RMSE [rad/s]	**0.032**	0.068
v_y persistence RMSE [m/s]	0.009	0.009
ω persistence RMSE [rad/s]	0.005	0.005
Skill vs. persistence, v_y	-2.0	$\mathbf{-1.4}$
Skill vs. persistence, ω	$\mathbf{-5.4}$	-12.6
Identified coefficients, last accepted set		
Front B	7.076	4.804
Front C	1.346	1.448
Front D	**1.009**	2.000†
Front E	-2.000	-2.264
Rear B	7.873	4.836
Rear C	1.383	1.467
Rear D	**1.002**	2.000†
Rear E	-1.024	-1.361

Source: `graphs/comparison/comparison_summary.csv`. Bold is the better of the two where the metric has a direction.

Lap times are recorded on the ROS wall clock while the simulator advances a fixed 0.033 s step faster than real time, and the two runs did not hold the same real-time factor, so they are reported as recorded and not bolded.

Max $|e_y|$ is the same acquisition transient in both runs—the pure-pursuit out-lap, before either identified model is in the loop—and agrees to 9 mm.

ITAE is not scale-free and the two runs differ in length, so IAE/T and $\mathrm{ITAE}/(T^2/2)$ are given beside it; all three are computed from the same exported error series.

The persistence rows are the reference baseline—next state equals last measured state—not a result.

Every value is a single run per configuration: no row carries a confidence interval, and the directional rows should be read as one paired observation, not as an estimate of a mean difference (Section VIII-C).

†On the box constraint $D \leq 2.0$. Only D has an external reference (the plant's $\mu = 1.05$) and is bolded on that basis; B, C and E are reported without one.

First, no coefficient of `Ours` rests on a box constraint, in any of its six accepted identifications: $B = 7.076/7.873$ inside $[4, 20]$, $C = 1.346/1.383$ inside $[1.2, 2.2]$, $D = 1.009/1.002$ inside $[0.4, 2.0]$ and $E = -2.000/-1.024$ inside $[-3, 1]$. The front E landing on a round -2.000 is the configured prior, not a bound: E has essentially no data support here (the lap reaches 2.2 ° of slip, and E shapes only the region around and past the peak), which is precisely the coefficient the frozen Tikhonov target of (12) exists to hold at its nominal value. Reading it as an identified quantity would be a mistake in either configuration; what the comparison shows is that the regularised fit parks it at the prior while the unregularised one moves it to -2.264 on no evidence, having started the run with E_f on the -3.0 bound. `Baseline-NN-MSE` puts D on its upper bound of 2.000 in two of its six accepted identifications—the first and the last, so the run both starts and ends on the bound—and E_f on its lower bound of -3.0 in the first. In between it wanders over $D_f \in [1.241, 2.000]$ without settling. A peak factor of 2.000 against a plant that realises 1.05 is not a 90 % error in a tire parameter so much as the box-edge signature of Section III-C: the coefficient is not identified, and nothing in a per-coefficient bounds check distinguishes that from a fit.

Second, the price `Ours` pays is in the linear range. Every slope here is (5) for an identified set and the closed form of Table II for the plant, both at $\bar{F}_z = 1$ and at the same static axle loads $F_{z,f} = 1373\,\mathrm{N}$, $F_{z,r} = 1274\,\mathrm{N}$, so the three are comparable. `Ours` gives 13.2 kN/rad front and 13.9 kN/rad rear against the plant's 20.4 and 19.0, i.e. 35 % and 27 % low, while `Baseline-NN-MSE` gives 19.1 and 18.1, 6.5 % and 4.6 % low. The understeer sign is correct in both ($l_r C_r > l_f C_f$), so both sets pass the stability gate of Section VII-A, and only `Baseline-NN-MSE`'s grip ceiling is a claim the plant cannot honour: (6) gives $2.00\,g$ for it against $1.01\,g$ for `Ours` and $\approx 1.0\,g$ measured.

The reason both effects can coexist is the slip-angle coverage of the lap. Over the identification samples of either run, $P_{99}(|\alpha|) = 0.038\,\mathrm{rad}$ (2.2 °) and $\max|\alpha| = 0.045\,\mathrm{rad}$, against the 12.0 ° at which the plant's curve first reaches its peak (Table II). The measured lap therefore constrains the slope and says almost nothing about the peak; only the synthetic rollout of Section V-B reaches up the curve, and only the warm start of Section V-E3 carries independent information about μ.

E. Peak friction

Figure 9 reports the error of the identified peak factor against the effective friction the plant reports per wheel, and Fig. 10 shows how it evolves over the run. With the surface static at $\mu = 1.05$, `Ours` returns $D_f = 1.009$ and $D_r = 1.002$ in *every* one of its six accepted identifications: the published peak factor is constant for the whole run, so its error is a constant offset and RMSE, MAE and maximum absolute error coincide at 0.041 and 0.048. The warm start's floor does not bind on any cycle here. `Baseline-NN-MSE` spans $[1.241, 2.000]$ front and $[1.358, 2.000]$ rear over its six, starting and ending on the bound, giving RMSE 0.762 and 0.772: 16–19× worse ($0.762/0.041 = 18.6$ front, $0.772/0.048 = 16.1$ rear).

One qualification belongs on the `Ours` column: the corrected pipeline *preserves* a physically valid peak factor rather than sharpening one. The settled value is its configured prior, measured on this vehicle, and the fit does not move it at all—six identifications, six identical coefficient sets to four decimal

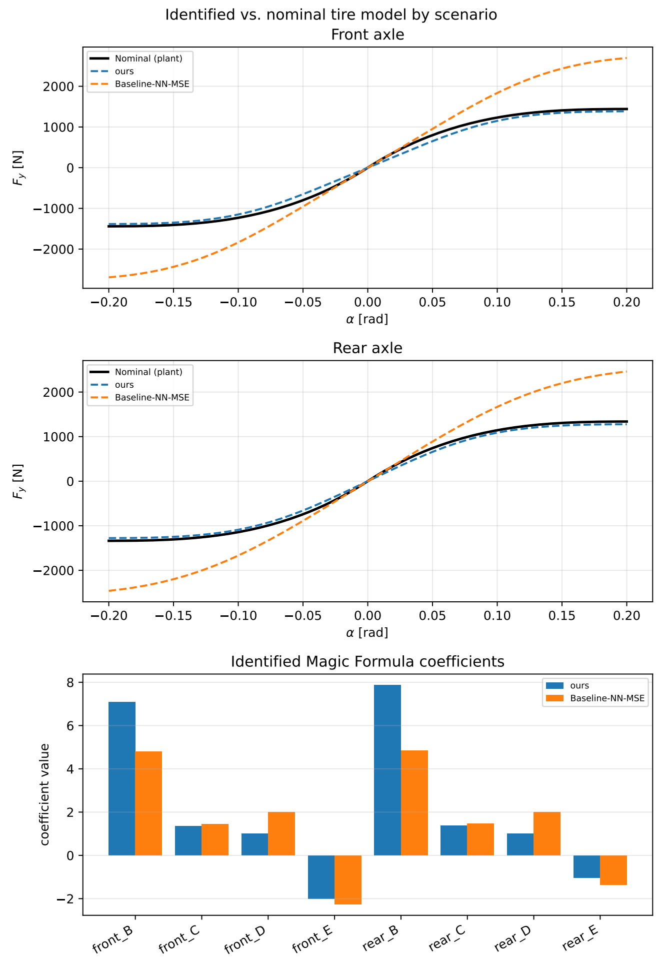


Fig. 8. Identified tire models against the plant. Top and middle: the plant's own curve (8) at the measured static load of each axle (black) against each run's last accepted identification, over $\alpha \in [-0.2, 0.2]$ rad. Bottom: the identified Magic Formula coefficients. `Baseline-NN-MSE`'s last accepted peak factor sits on its box constraint $D = 2.0$ on both axles, so its curve carries roughly twice the plant's force at the edge of the plotted range; `Ours` tracks the plant's peak and undercuts its initial slope.

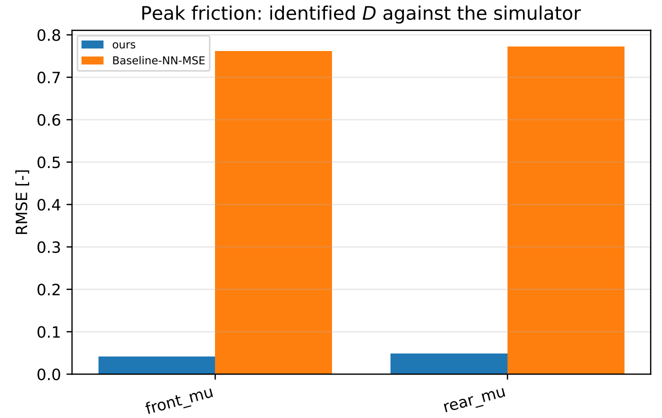


Fig. 9. Peak-friction error per axle against the effective coefficient the simulator reports per wheel ($\mu = 1.05$, static for both runs). The corrected configuration's error is an order of magnitude smaller on both axles (0.041 and 0.048 against 0.762 and 0.772 RMSE); the inherited configuration's error is not a mis-estimate but a coefficient parked on its upper bound.

places. What the comparison shows is that the same data, the same rollout and the same box constraints drive the inherited configuration onto the bound instead, and that the difference is worth 0.99 g of claimed grip.

F. Axle lateral force

Figures 11–13 score the identified curve at the measured (α, F_z) against the plant's per-wheel lateral force. The ordering reverses across axles: `Ours` is better at the rear (40.8 N against 50.1 N RMSE, R^2 0.979 against 0.970) and worse at the front (70.7 N against 45.9 N, R^2 0.944 against 0.978). The error histograms (Fig. 12) are centred in both cases and differ in width and tails: `Baseline-NN-MSE`'s maximum absolute error is 215 N front and 223 N rear against 138 N and 85 N for `Ours`, so its lower front RMSE is bought with a tail 1.6× heavier at the front and 2.6× at the rear. Where the force is largest each model errs in the direction its initial slope predicts—`Ours` short, the inherited configuration long.

This is the linear-range deficit of Section VI-D showing up as force error, and it is the one axis on which the inherited configuration is better. At 2.2° of slip both curves are far below their peaks, so a peak factor on its box constraint costs nothing here while a 35 % slope deficit costs 71 N of front-axle RMSE. The metric that separates the two models is therefore not the force at the slip angles the lap visits but the friction ceiling those forces do not probe—which is precisely what a controller planning a speed profile consumes.

G. One-step state prediction

Level 1 of the validation protocol (Section V-G) is reported in Fig. 14 against a persistence baseline—next state equals last measured state—written by the same benchmark node. `Ours` predicts yaw rate at 0.032 rad/s RMSE against `Baseline-NN-MSE`'s 0.068 (R^2 0.950 against 0.788), and lateral velocity marginally worse (0.027 m/s against 0.022, R^2 0.988 against 0.992). The error histograms (Figs. 15 and 16) show the yaw-rate difference as tail mass: maximum absolute error 0.452 rad/s for `Ours` against 1.030 rad/s.

Neither model beats persistence at the 0.035 s model step: the skill scores $1 - \mathrm{RMSE}/\mathrm{RMSE}_{\mathrm{pers}}$ are negative for both configurations and both signals, worst at -12.6 (`Baseline-NN-MSE`, ω) and best at -1.4 (`Baseline-NN-MSE`, v_y). We report this rather than the R^2 alone because R^2 against a slowly varying signal is close to uninformative—a one-step metric on a smooth trajectory rewards not moving, and over one 0.035 s step the plant's own states barely move ($\mathrm{RMSE}_{\mathrm{pers}}$ is 0.009 m/s and 0.005 rad/s). One-step accuracy is consequently not evidence for either tire model here; the axle-force and closed-loop comparisons are. Figure 17 shows the two state traces against their own ground truth over the whole run.

H. Closed-loop tracking

Level 3 of the validation protocol completes on the regenerated trajectory. Both runs finish three timed laps with

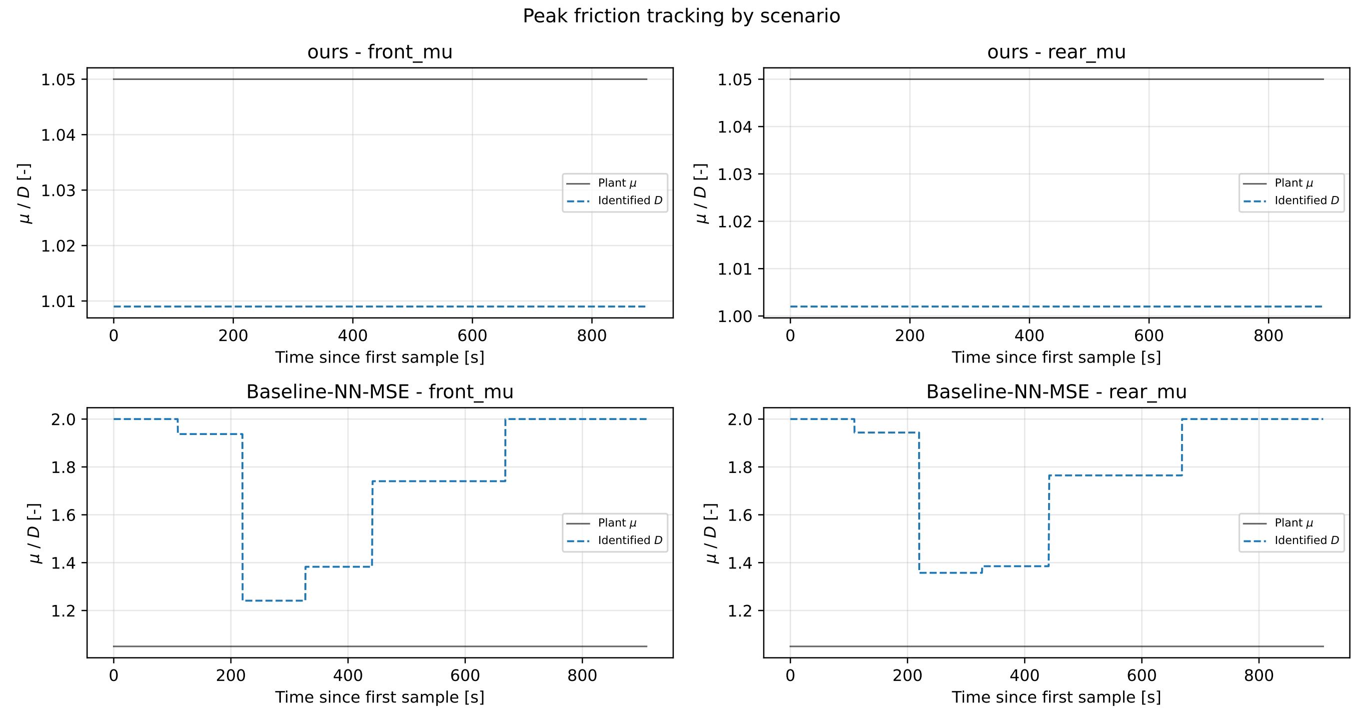


Fig. 10. Peak friction over the run, one row per configuration: the plant's effective μ (black) against the identified peak factor D published at each of the six accepted identifications. The surface is static, so every departure from the black line is estimator error. `Ours` is a flat line about 0.04 below the plant on both axles—the same coefficients every cycle—while `Baseline-NN-MSE` starts and ends on the $D \leq 2.0$ box constraint and does not settle in between.

zero emergency halts and the same finite-state sequence—bootstrap pure pursuit, running pure pursuit, handover, running MPC—with one transition of each kind and no fallback to pure pursuit after handover. The sequence is read off each run's exported state timeline (`graphs/control/*/fsm_state_timeline.csv`), which records the handover at $t = 81.4$ s for `Ours` and $t = 66.0$ s for `Baseline-NN-MSE` and no state change thereafter; the aggregated `transitions` column of the cross-run summary is not populated and is omitted from Table VI. Under MPC, which is where the identified model enters the loop, `Ours` tracks at 0.092 m RMS lateral error against 0.177 m, a 48 % reduction, with maximum 0.490 m against 0.696 m (Fig. 18). Overall ITAE is 10362 against 17651 (Fig. 20). ITAE is $\int t|e_y|\,\mathrm{d}t$ and is neither scale-free nor comparable across runs of different length, and these two runs are 591.4 s and 564.1 s long, so it is reported here beside two normalised forms computed from the same samples: mean absolute error IAE/T, 0.085 m against 0.132 m, and $\mathrm{ITAE}/(T^2/2)$, 0.059 m against 0.111 m. All three order the runs the same way and at the same magnitude, so the normalisation does not carry the result. The ITAE split confirms that the whole difference is in the MPC phase: the pure-pursuit phases, which do not use the identified model, give 211 and 122 respectively, ordered the other way and two orders of magnitude smaller. The signed error distribution (Fig. 19) shows the mechanism as a narrower distribution about a comparable centre—sample standard deviation 0.279 m against 0.301 m, mean 0.014 m against 0.039 m.

The QP cost is unchanged: RMS solve time 0.413 ms against 0.390 ms, worst case 3.81 ms against 4.06 ms, both two orders of magnitude below the 50 ms control period, and the two orderings disagree, which is what "unchanged" means here. The identified coefficients change the model the QP carries, not its size.

Lap time is not read as a result. Timing runs on the ROS wall clock while the simulator advances its fixed step faster than real time, and the ratio between the two differed between the runs, so the recorded lap times in Table VI are not simulated lap times and their 4–6 % spread is not a vehicle-performance difference.

I. Baseline pipeline versus this work

Table VII places the pipeline as published [6] beside the pipeline as it stands here, on the axes that changed. The residual model, the data budget, the four-stage loop and the steady-state force recovery are inherited unchanged; what differs is the configuration of the virtual-data step, the regularisation topology, the measurement chain, and the contract at the output. Table IV's comparison is a subset of these axes: the rollout correction of Section V-B is active in both runs of Section VI-C, so that section measures the remaining four settings and not the rollout.

J. Peak-friction warm start on a synthetic plant

The friction warm start of Section V-E3 is evaluated on a synthetic single-track plant with brush tires rather than in CARLA, because only a synthetic plant supplies a μ that is known exactly and an excitation level that can be swept

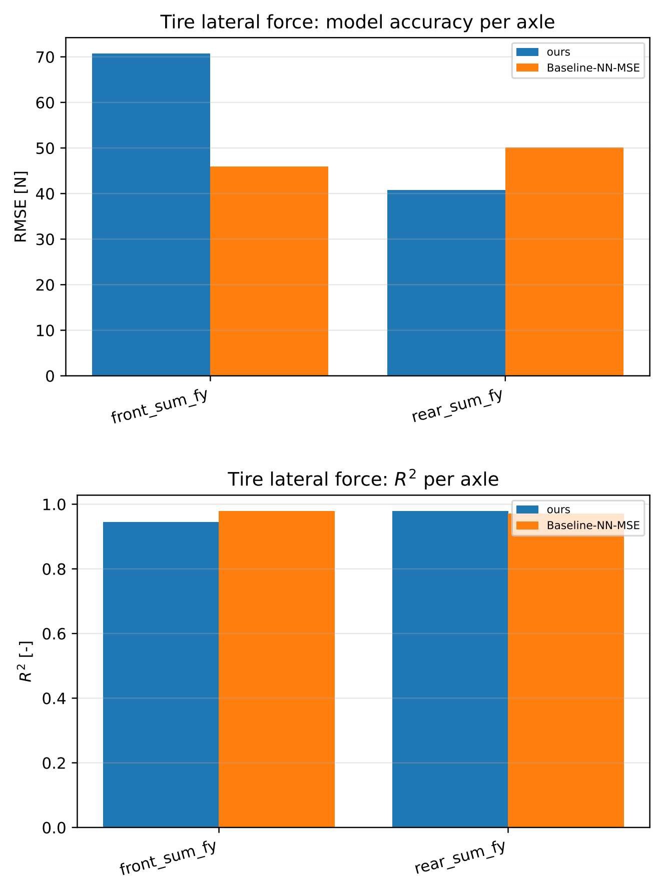


Fig. 11. Axle lateral force against the plant's per-wheel telemetry, per axle: RMSE (top) and R^2 (bottom). The ordering reverses between axles, and this is the one family of metrics on which the inherited configuration is better—at the 2.2 ° of slip the lap reaches, a slope deficit costs force and a wrong peak factor costs nothing.

independently of it. The plant is driven by a swept-sine steering input for 40 s at 50 Hz with true $\mu = 1.05$ and $C_\alpha = 12.1F_z$, and the steering amplitude sets how far up the tire curve the manoeuvre reaches. Table VIII reports what each estimator returns.

At 45 % utilisation—a level an ordinary lap reaches—the utilisation bound is 55 % low while the brush fit is within 0.2 %. Across true $\mu \in \{0.7, 1.05, 1.4\}$ the fit stays within 5 % and C_α within 10 %; with odometry and IMU noise injected, μ stays within 12 %. Below the 40 % floor the gate suppresses the fit and reports the bound, so the estimator's failure is visible rather than silent. On the CARLA runs above the lap's utilisation leaves the released estimate below the configured prior, so the floor never binds and the warm start does not move D on any of the six cycles (Fig. 10); its contribution there is that it cannot lower D, not that it raises it.

K. Identification cost

Section V-A4's claim is that the temporal residual is affordable on the hardware a vehicle carries; Table IX is the measurement, on a synthetic 30 s lap and the same six co-identification iterations throughout. The per-stage rows and the end-to-end totals were measured separately and are not a decomposition of each other: the stage rows reconstruct 156.2 s of the 181.4 s before and 21.1 s of the 20.9 s after, so the residual 25.2 s of fixed cost visible before is not visible after. We report both as measured rather than reconciling them by attribution. Every step was verified against the implementation it replaced before being kept: the convolutional and recurrent paths of Fig. 5(c) agree to $< 10^{-9}$ in double precision (pinned by a regression test), the rewritten physics-informed loss is bit-identical, and the full six-iteration loop lands on the same Pacejka coefficients to four decimal places—the remaining difference is float32 reordering over 1200 optimiser steps.

Two consequences follow. At 20.9 s the identification fits inside the 30 s re-identification period of Section V-F, so the temporal residual is a deployable option rather than an offline

TABLE VII
THE BASELINE PIPELINE [6] AGAINST THIS WORK, ON THE AXES THAT DIFFER.

Axis	Baseline [6]	This work
Platform	1:10, 3.7 kg, $L = 0.32$ m	Full scale, 269.8 kg, $L = 1.53$ m
Rollout speed	clip($\overline{v_x}$, 2, 4) m/s	From the measured lap, floored at 15 m/s
Reachable μ	≈ 2.0 (adequate)	0.43 under the baseline clip; ≥ 1.2 after correction
Sweep amplitude	Fixed 0.4 rad	$1.5P_{99}(\lvert\delta_{\mathrm{obs}}\rvert)$, clipped to $[\delta_{\min}, \delta_{\max}]$
Regularisation	None; each fit is the next start point	Tikhonov to a *frozen* prior; start point still tracks
Steering signal	Command	Achieved road-wheel angle
Mass, geometry	Configuration file	Measured static wheel loads
Friction prior	—	Per-axle brush fit of (C_α, μ) from IMU and odometry, gated; P_{99} of $\lvert a\rvert/g$ as fall-back; always a floor
Solver	Trust-region, single start	Multi-start, plus simplex and global options
Output contract	Per-coefficient box constraints	Gates on derived quantities at the controller interface
Residual objective	One-step MSE	MSE plus quasi-steady balance, left/right symmetry and input-gradient penalties, (9)
Residual inputs	State, command	*inherited* plus axle slip angles (Sec. V-A1)
Residual model	MLP, 58 parameters	*inherited* as default; S4D temporal residual shipped
Residual cost	Not reported	S4D identification 181.4 s → 20.9 s, exact (Table IX)
Data budget	30 s, one controller-free lap	*inherited*
Force recovery	Quasi-steady yaw balance	*inherited*
Ground truth	Steady-state manoeuvre in a large space	Simulator per-wheel force, slip and load

Rows marked *inherited* are unchanged and are listed so that the extent of the modification is unambiguous.

TABLE VIII
PEAK FRICTION RECOVERED FROM A SYNTHETIC BRUSH-TIRE PLANT (TRUE $\mu = 1.05$), AGAINST EXCITATION.

δ amplitude	Grip utilisation	Utilisation	Brush fit (14)
0.05 rad	45 %	0.473	**1.048**
0.07 rad	63 %	0.658	**1.048**
0.09 rad	80 %	0.834	**1.048**
0.13 rad	100 %	1.045	**1.048**

The utilisation estimator tracks the grip the manoeuvre *used*; the brush fit of (14) recovers the peak from the curvature of $F_y(\alpha)$ and is therefore flat in excitation.
Below 40 % utilisation the brush fit is gated off and the utilisation bound is reported instead.

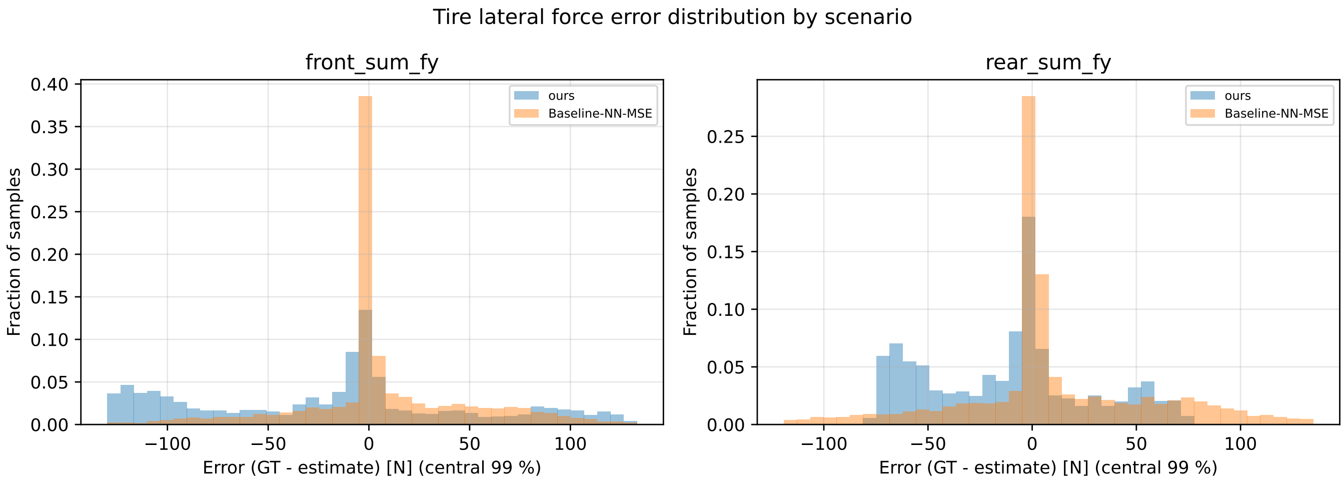


Fig. 12. Distribution of the axle lateral-force error (truth minus estimate), shared bin edges over the pooled central 99 % and normalised to the sample count of each run. Both are centred; the inherited configuration buys its lower front RMSE with the heavier tail (maximum absolute error 215 N against 138 N front, 223 N against 85 N rear).

TABLE IX
COST OF ONE S4D IDENTIFICATION ON A SYNTHETIC 30 S LAP.

Stage	Device	Before	After
Training epoch: scan → kernel (10)	GPU	125.7 ms	26.7 ms
Training epoch: physics-loss hoist and fusion	GPU	26.7 ms	16.6 ms
Stage-2 rollout, 500 sequential steps	GPU → CPU	898 ms	204 ms
Identification, 6 iterations	mixed	181.4 s	20.9 s

The optimisations are exact: no epoch count, iteration count, early-stopping rule or loss weight was changed, and the identified coefficients agree to four decimal places.
The first three rows are per-stage micro-benchmarks of the named stage; the last row is a separate end-to-end measurement of the whole six-iteration loop and includes everything the micro-benchmarks exclude—data preparation, the per-iteration network re-initialisation and the six Pacejka fits.
The two therefore do not sum: at 1200 epochs and six rollouts the stage rows account for 156.2 s of the 181.4 s before and 21.1 s of the 20.9 s after. The 8.7× is the ratio of the two end-to-end measurements.

one; at 181.4 s it was not. And the device split is not incidental: the stage-2 rollout is strictly sequential and single-sample, so it is 4.4× *faster* on a CPU, while full-batch training over ≈ 3000 windows remains 2.4× faster on the GPU. A pipeline pinned to one device pays one of those two penalties.

VII. INTEGRATION WITH THE CONTROLLER

The identified model leaves the pipeline through one interface. A nonlinear MPC path-tracking controller (acados [31], partial-condensing HPIPM [32], horizon $N = 20$ at a 20 Hz control period) carries the same single-track model as the identifier and consumes Φ_p directly: the eight Pacejka coefficients enter the prediction model's axle forces through (4), and nothing else about the controller changes when a new identification arrives. Coefficients are pushed over a ROS 2 service at the end of each identification cycle, so the controller's model tracks the plant at the re-identification rate. We claim no contribution to the controller design; it appears here as the consumer that defines what an admissible identification is.

A. Admissibility gates

A coefficient set can lie strictly inside every box constraint and still be unusable. We observed three distinct ways, each requiring a gate on a *derived* quantity rather than on a coefficient:

1) **Grip ceiling versus trajectory demand.** The controller computes $\max_k(v_{x,k}^2|\kappa_k|)$ over the speed-clamped reference and rejects any identification whose $a_y^{\max}$ from (6) falls below it. On the earlier trajectory the degenerate fit reported a 0.40 g ceiling against a 1.22 g demand and produced 41.89 m of cross-track error at 27 m/s, against 0.03 m for a physically plausible set ($D = 1.5$) on the same trajectory, at the same speed and through the same controller. Both figures come from an offline closed-loop harness—20 s on the reference with the handover at $t = 5$ s—and not from a controlled pair of simulator runs; they are quoted as the observation that motivated the gate, not as a result. They are not confounded with the transport-delay correction below, which was already in place when they were measured. The same identified set on the same trajectory gives 0.16 m at 11.1 m/s and 0.43 m at 15 m/s, which is the v^2 scaling the Introduction describes. The gate is one-sided by construction and only catches a ceiling that is too low: `Baseline-NN-MSE`'s 2.00 g (Section VI-D) passes it, since it exceeds every demand the reference makes.
2) **Understeer sign and critical speed.** A set with $l_f C_f > l_r C_r$ whose v_{crit} falls inside the controller's speed range makes the prediction model open-loop unstable, and the resulting $\rho(A_d) > 1$ is raised to ρ^N in the condensed QP. An observed set gave $C_f = 37031$ N/rad against $C_r = 11691$ N/rad, i.e. $v_{\text{crit}} = 14.5$ m/s at the measured geometry and mass of Table I, inside the controller's speed range on every trajectory in this paper.

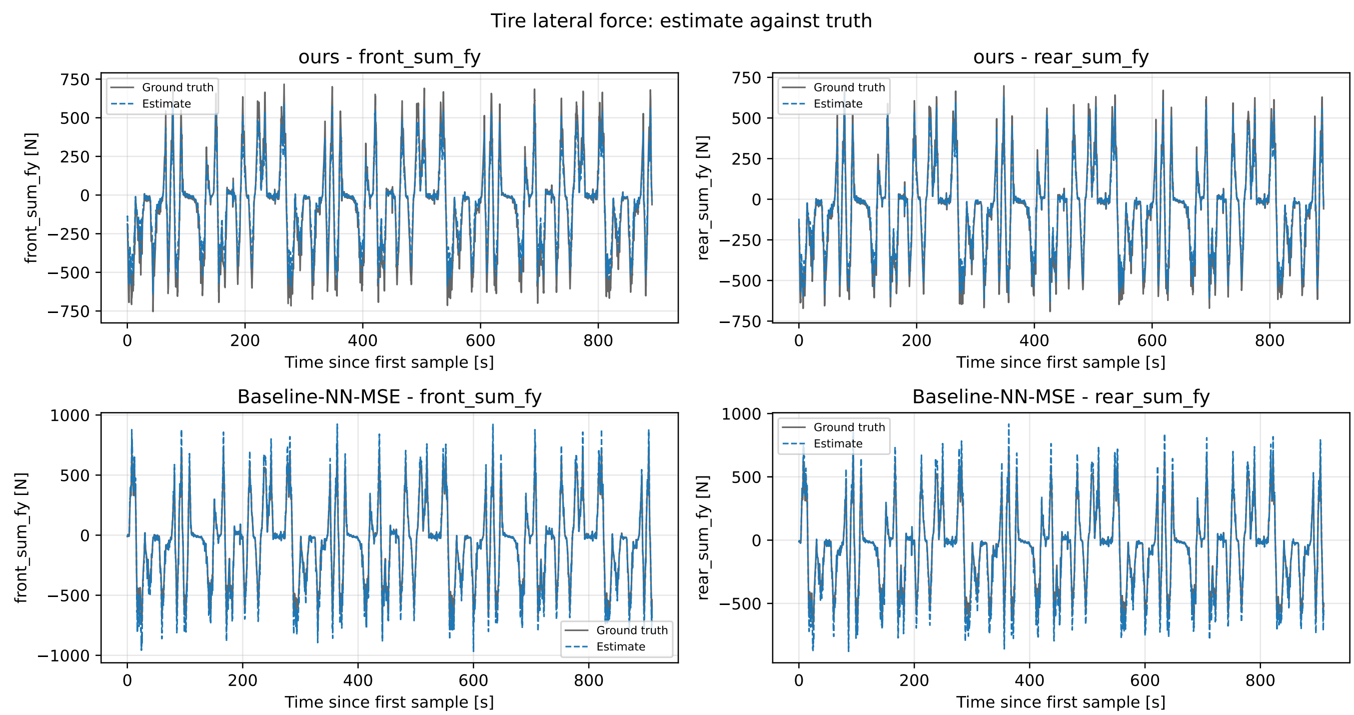


Fig. 13. Axle lateral force over the run, one row per configuration. The two runs are separate laps and share only $t = 0$, so each estimate is drawn against its own ground truth. Where the force is largest each model errs in the direction its initial slope predicts—the corrected configuration short, the inherited one long.

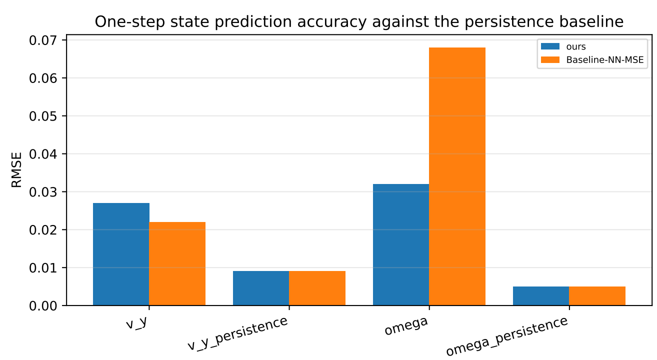


Fig. 14. One-step prediction RMSE for v_y and ω against the persistence baseline of the same run. Neither identified model beats persistence at the 0.035 s model step.

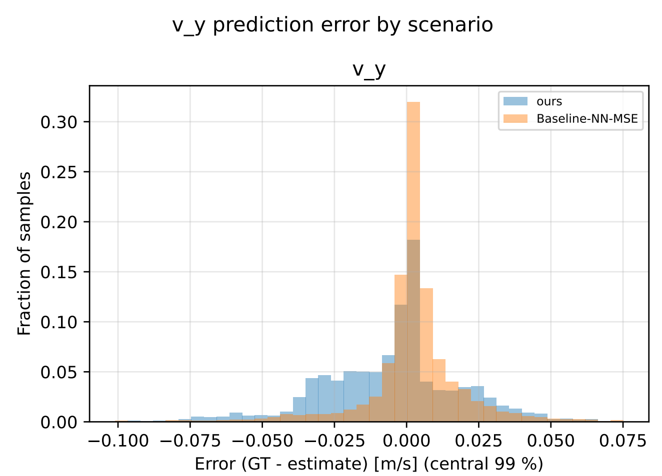


Fig. 15. Lateral-velocity prediction error (truth minus estimate). The two configurations are indistinguishable on this channel (0.027 m/s against 0.022 m/s RMSE); the yaw channel, not this one, is where they differ.

3) **Integration stiffness.** Even a stable set can be stiff relative to the control period: a single explicit RK4 step at $dt = 0.05$ s gave $\rho(A_d) = 547$ at 5 m/s for one identified set. The model sub-steps adaptively, sizing the sub-step from the model's own fastest lateral mode $|\lambda| \approx (C_f + C_r)/(mv_x) + (l_f^2 C_f + l_r^2 C_r)/(I_z v_x)$ so that $|\lambda| h \leq 1.5$, restoring $\rho \approx 1.000$ across the speed range at 2.3–6.8 ms per 100-stage horizon.

The gates are the contract between identification and control: an identification that passes is a model the controller can be held responsible for, and one that fails is reported as unidentified rather than silently deployed. Two further controller-side changes were required before any identified model could be evaluated at full-scale speeds and are recorded for completeness rather than as identification results: reference speeds are clamped at ingest, and the solved state is rolled forward by the measured transport delay before each solve, which took cross-track error at 27 m/s from 130.55 m to 0.38 m.

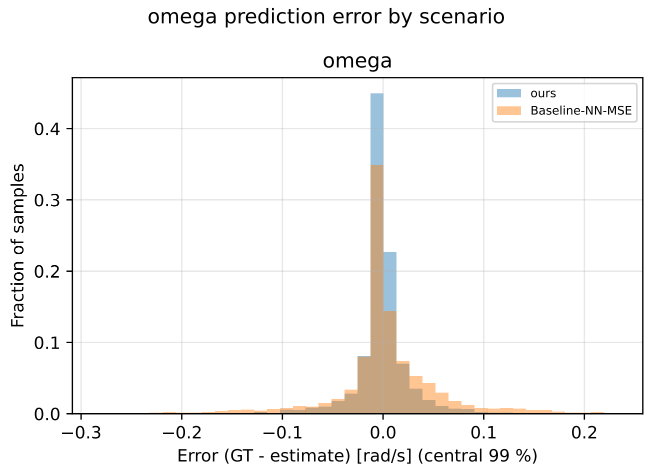


Fig. 16. Yaw-rate prediction error (truth minus estimate). The difference between the configurations is tail mass, not centre.

VIII. Discussion and Limitations

A. What generalises

The reachable-friction bound (11) is kinematic: it involves no tire parameter, no simulator-specific quantity and no property of the residual model, so it applies to any implementation of the virtual-steady-state-data strategy of [6], in simulation or on hardware. Its practical content is a design rule—*the rollout must be configured to demand at least the friction one intends to identify*—and, where it cannot, D should be reported as unidentified rather than as fitted. The diagnostic signature, coefficients resting exactly on box constraints, is likewise generic and should be checked and logged by any bounded tire-fitting routine, since it is otherwise indistinguishable in the output from a successful fit.

The insufficiency of per-coefficient box constraints for deployment admissibility is also platform-independent. Both failures we encountered—a set that satisfies every box constraint while being degenerate, and a set that satisfies every box constraint while being pairwise oversteering and unstable within the controller's operating range—are properties of the parameterisation, not of CARLA. Deep Dynamics [19] constrains coefficients inside the network; we argue that a second, derived-quantity check belongs at the interface where the model is handed to a controller.

B. What does not generalise

Three results are properties of this plant and must not be read as statements about tires. First, the numerical tire values: the plant is a PhysX wheeled-vehicle model with a configured friction coefficient, so Table II describes *that* plant, and agreement with it is evidence that the identifier recovers the plant it observes, not evidence about real tire behaviour. Sagmeister *et al.* [28] show that sensitivity to model fidelity is largest near the acceleration limits, which is where this study operates, and R-CARLA [27] exists because CARLA's native dynamics are a known weak point for racing-grade work; hardware replication is therefore necessary before any claim about identification accuracy on a real vehicle. Second, the gap between configured wheel friction (1.5) and realised peak axle friction (1.00–1.05) is an artefact of the server multiplying the configured wheel coefficient by the road surface's own, a measured constant 0.70 on this map; we report it only because an identifier—or a reviewer—tuned to converge on the number in the configuration file will converge on the wrong number, and because the simulator publishes the effective value per wheel, so nothing forces that mistake. Third, the 0.76 achieved/commanded steering ratio arises from CARLA's Ackermann controller and this vehicle's steering configuration—the lesson, identify against the achieved actuator state, is general and standard practice; the magnitude is not.

C. Limitations

1) **One run per configuration, so no metric in Table VI carries an uncertainty.** Every value there is a point estimate from a single 591.4 s and a single 564.1 s run. The 0.092 m against 0.177 m RMS tracking difference, in particular, has no confidence interval attached and no evidence that it exceeds run-to-run variation, which is not measured here. Nothing about the experiment prevents repetition—the sweep is one command and the scenario file already carries the per-arm parameter sets—so this is a gap in the evidence rather than in the method.
2) **Single surface, vehicle and trajectory.** All results come from one vehicle configuration on one map with the friction held static at 1.05. The runtime friction override supports a surface-change adaptation experiment—arguably the strongest argument for on-track identification—but it is not performed here.
3) **The friction estimator's excitation sweep is synthetic** (Table VIII). The synthetic plant was chosen because μ is known there and excitation can be swept independently of it. On the CARLA runs the warm start is exercised only at the one excitation level the lap happens to supply, and only through the floor it puts under D on the first identification, so its excitation dependence is not verified against the simulator's per-wheel forces.
4) **The two configurations differ in four settings at once** (Table IV), so Section VI attributes nothing to the residual architecture, the loss, the solver or the warm start individually; in particular we do not claim to reproduce the S4 improvement of [20]. The excitation-aware rollout—the paper's central contribution—is active in *both* arms, so the closed-loop comparison of Section VI-H is a comparison of the four secondary settings and is not evidence for the identifiability correction. The evidence for that correction is the rollout-speed sweep of Section VI-B. What is claimed about the S4D residual on its own is its cost and the exactness of the reduction (Table IX), a statement about the implementation rather than about the tire it identifies.
5) **Longitudinal dynamics are out of scope.** As in the baseline only lateral dynamics are identified; combined slip and load transfer are neglected, and the simulator's

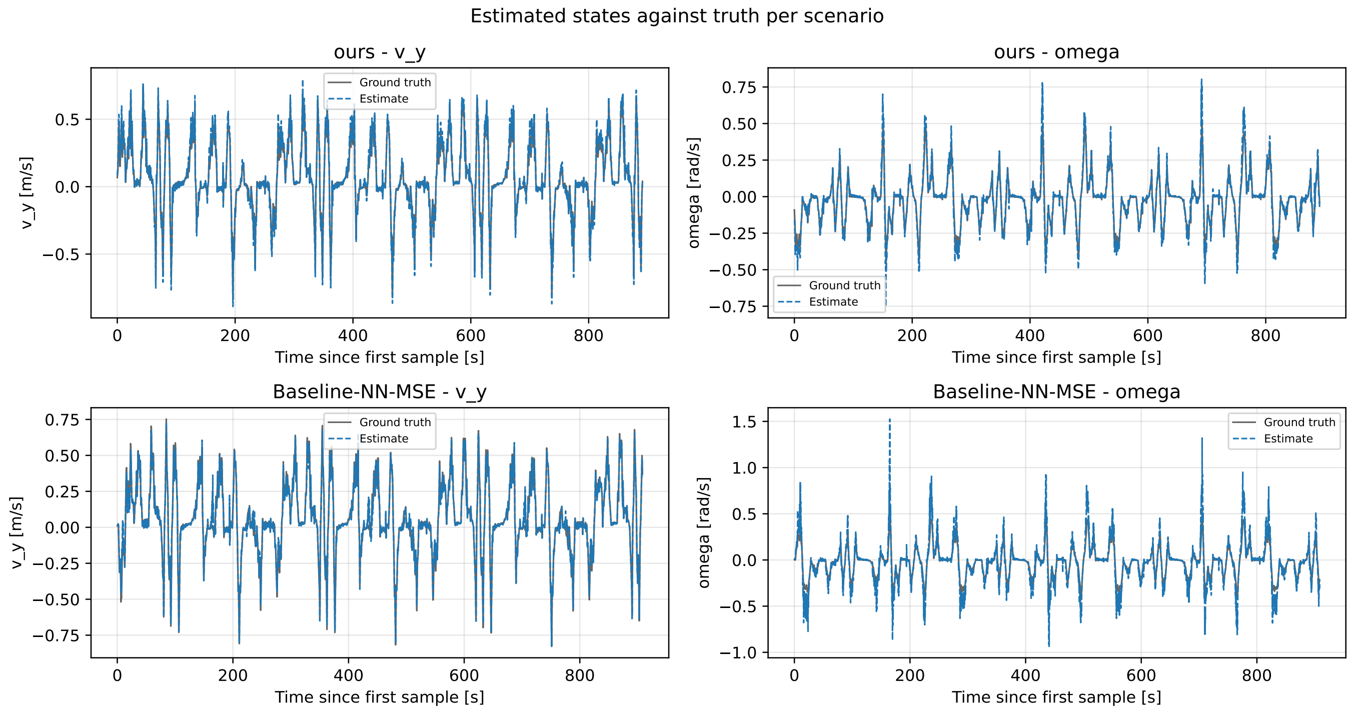


Fig. 17. Estimated states against truth over the run, one row per configuration, 591 s and 564 s of driving. At this compression the traces overlie their ground truth almost everywhere, which is the point: a one-step metric on a smooth racing line is close to uninformative, and the visible departures are the yaw-rate excursions that separate the two configurations in Fig. 16.

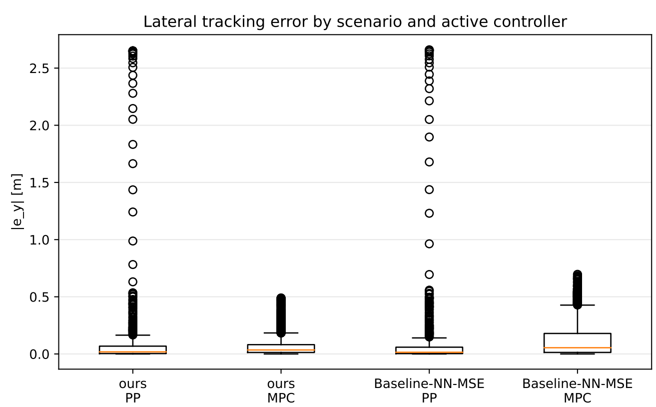


Fig. 18. Absolute lateral tracking error by configuration and active controller. The pure-pursuit boxes cover the acquisition phase before handover and do not use the identified model; the MPC boxes do.

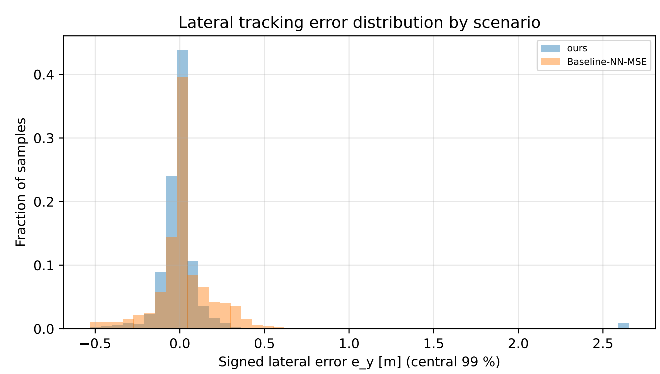


Fig. 19. Signed lateral tracking error over the whole run. The corrected configuration's distribution is narrower about a comparable centre—sample standard deviation 0.279 m against 0.301 m, mean 0.014 m against 0.039 m—so the improvement is in spread, not in bias.

longitudinal wheel force channel is drivetrain torque rather than a contact-patch force, so it is unusable as ground truth for a longitudinal extension without further work.

IX. Conclusion and Future Work

This paper studied the identifiability of a learning-based on-track Pacejka identification pipeline [6] at full vehicle scale, using a CARLA simulation architecture that exposes the plant's own per-wheel forces as ground truth. Transplanted unchanged from a 1:10 platform, the pipeline fails in a specific, diagnosable and correctable way. The failure is not a solver, bounds or tuning problem: it is a loss of excitation created by the pipeline's own virtual-data step, whose reachable friction $\mu_{\text{reach}} = v^2\delta_{\text{sweep}}/(gL)$ depends on the rollout configuration and not on the tire. A rollout speed appropriate at 1:10 caps μ_{reach} at 0.43 at full scale, below the plant's peak, so the peak factor is not identifiable from the synthetic data and the bounded optimiser resolves the ambiguity on a box edge.

Correcting this—together with an extrapolation bound on the steering sweep, a frozen regularisation target, identification against the achieved rather than the commanded road-wheel angle, measured mass and geometry, and a friction warm start identified from the curvature of the axle force curve and

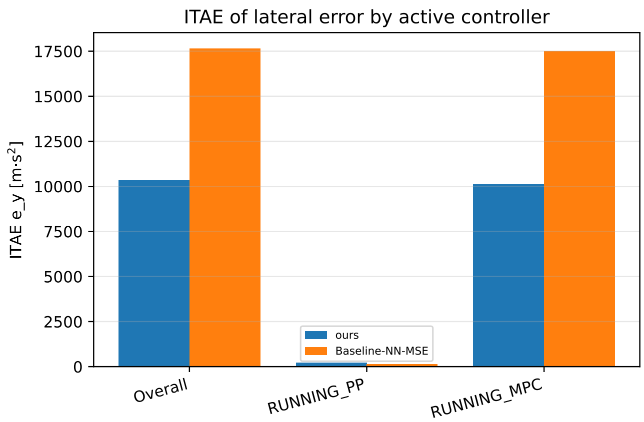


Fig. 20. ITAE of the lateral error, overall and split by the active controller. The pure-pursuit phase is ordered the other way and is two orders of magnitude smaller.

applied only as a floor—produces identifications with no railed coefficient and a peak factor of 1.009/1.002 against a plant realising 1.05, with the correct understeer sign. Run against the inherited configuration over three timed laps on a static surface, it holds that peak factor to 0.041 and 0.048 RMSE where the inherited configuration starts and ends on its 2.0 box constraint, and halves the MPC's lateral tracking error, from 0.177 m to 0.092 m RMS, at unchanged solver cost. The inherited configuration is better on front-axle force and on lateral-velocity prediction, because the lap reaches $2.2\,^\circ$ of slip and a wrong peak factor is not observable there—which is the same identifiability statement seen from the metric side. We further argue that per-coefficient box constraints are not a sufficient admissibility criterion for handing a model to a model-based controller, and place gates on derived physical quantities at that interface instead; the earlier reference trajectory, which demanded $1.22\,g$ from a vehicle sustaining $1.0\,g$, was regenerated because that gate refused it rather than loosened for it.

A. Future work

a) Immediate:

1) Separate the four settings of Table IV. The comparison reported here moves the residual architecture, the objective, the solver and the warm start together, so it establishes that the shipped configuration is better on the metrics that matter to a controller without saying which change earns it. The runtime objection to the temporal arm is removed (Section VI-K).
2) Excite the peak. Every result here is obtained at $P_{99}(|\alpha|) = 2.2\,^\circ$ against a plant whose curve peaks at $12.0\,^\circ$, which is why a peak factor on its box constraint costs almost nothing in axle force. A trajectory or an injected manoeuvre that reaches further up the curve would make the two models separable on force as well as on friction.
3) Run the noise sweep of [6] on this platform using the bridge's odometry noise model, with a nonlinear least-squares identification of the same data as the comparison arm.
4) Cross-check the friction warm start against the simulator's per-wheel forces across a pace sweep, including the σ_μ/μ gate boundary and the IMU-versus-kinematics cross-check that guards the mass and inertia assumptions. It is so far validated against a synthetic brush-tire plant, where μ is known exactly (Table VIII).
5) Report lap time on the simulator's own clock. Lap timing currently runs on the ROS wall clock against a faster-than-real-time simulator, which makes lap time unusable as a comparison metric (Section VI-H).
6) Measure I_z rather than assuming it (Section A).

b) Beyond:

1) **Excitation-aware data collection.** Equation (11) inverts into a requirement on the manoeuvre. A controller that deliberately inserts bounded slalom or ramp-steer excitation when the identifier reports insufficient slip-angle coverage would move the pipeline from passive on-track identification toward active, safety-bounded on-track experiment design—which is optimal input design [52] under a safety constraint, and is where the literature for this step already exists.
2) **Uncertainty-aware output.** The gates of Section VII-A are binary. Reporting a posterior over the coefficients—the Bayesian variational path already implemented alongside the deterministic solvers—would let the controller weight the model by its confidence rather than accept or reject it.
3) **Hardware replication.** The decisive test of every claim in Section VIII is a physical Formula Student vehicle. The identification code is unchanged between simulation and hardware; what changes is the availability of ground truth, which is what makes the simulation study a necessary precursor rather than a substitute.

Appendix A
Assumptions and Reproducibility Notes

The following are assumptions or design choices rather than measured facts. They are listed explicitly so that a reader can judge how far each result depends on them, and so that each can be discharged by a specific measurement.

A1. Yaw inertia I_z

$I_z = 51.1\,\mathrm{kg\,m^2}$ is carried in the model file and is *not* measured. It enters (2) directly and therefore scales the yaw-rate residual the network is trained on, and it enters the stiffness-adaptive sub-stepping of Section VII-A through $l_f^2 C_f + l_r^2 C_r$ over I_z. *How to discharge:* apply a known yaw moment (a step steer at constant speed with the resulting axle forces read from the force feedback topic) and fit I_z from the measured $\dot{\omega}$; or read the value the physics engine is using directly from the actor's inertia tensor.

A2. Centre-of-gravity split

$l_f = 0.738142\,\mathrm{m}$, $l_r = 0.795362\,\mathrm{m}$ are taken from the controller configuration and confirmed by the vehicle owner.

The measured static load split (51.2/48.8 front/rear) implies a marginally different split than the geometry does (51.9/48.1) — a 1.4 % disagreement. We have not resolved it, and it is below the level at which any conclusion in this paper would change.

A3. CG height and load transfer

$h_{\text{cg}} = 0.3\,\text{m}$ is carried in the model file and unused: the single-track model of Section III-A neglects load transfer, following the baseline. Normal loads are static. On a full-scale vehicle at $1\,g$ this is a stronger approximation than at 1:10, and it is a candidate explanation for part of the residual the network absorbs.

A4. Steering-angle sign and units

`feedback/steering_angle` is published in degrees and is treated as the front road-wheel angle, positive left, matching the ROS body frame. It was verified to agree with the front-left wheel joint state to within 0.5 %. Left and right road wheels are not distinguished; the single-track δ is taken as this single reported value.

A5. Steady-state force recovery

Equation (7) is not a force measurement — it is fully determined by $v_x\omega$ — so it misattributes transients. It is applied only to the *synthetic*, quasi-steady rollout, where the steady-state assumption is constructed rather than hoped for. Its adequacy was nonetheless checked on measured data against the simulator's own axle force (correlation 0.993, slope 0.94); we treat that as validating the relation, not as validating a steady-state assumption on recorded laps.

A6. The residual network is not the plant

The corrected model $\hat{\mathbf{x}}_{k+1} + e_k$ is queried by the rollout at inputs the network has only partially seen. The extrapolation bound of Section V-C limits but does not eliminate this: the sweep still reaches $1.5\times$ the observed 99th percentile of $|\delta|$. The choice of 1.5 is empirical, selected because it was sufficient to reach the tire's peak at the corrected rollout speed while removing the observed drift; it is not derived.

A7. Simulator determinism and timing

CARLA runs in synchronous mode with a 0.033 s physics step, but sim time advances faster than wall time (observed $\approx 1.9\times$ on our hardware). All identification data is stamped from the simulation clock.

A8. Software configuration of record

Identification and control workspace: ROS 2 Humble, Python 3.10 (required — ROS 2 Humble ships CPython 3.10 ABI extensions only), PyTorch with CUDA on an RTX 2080-class GPU. The bridge is built and sourced as a separate overlay on top of that workspace, so that its message packages (`sim_manager_msgs`) resolve for the ground-truth forces; CARLA server 0.9.16 / Unreal Engine 4.26; **acados 0.5.5** with partial-condensing HPIPM. Identification runs at 20 Hz with a 30 s buffer, six co-identification iterations, 200 training epochs, full-batch Adam at 5×10^{-4}. The one-step model is discretised at $T_s = 0.035\,\text{s}$ (`nn_params.yaml: sample_dt`), matched to the server's 0.033 s step and not to the 50 Hz acquisition timer (Section IV-E).

Random seeds. `nn_train` seeds PyTorch and NumPy at 42 for the single-member architectures used here (42+i per ensemble member). `pacejka_solver.seed` is `null` in both shipped parameter sets, so the multi-start jitter and—for `Ours`—the differential-evolution population draw from an unseeded generator: the reported identifications are reproducible in distribution but not bit for bit. The sweep of Table V is seeded explicitly and is bit-reproducible.

Appendix B
Reference Implementation

The identification package, the ROS 2 bridge and the controller stack used in this paper are available as `On-Track-SysID`, extended from [6] (upstream: https://github.com/ForzaETH/On-Track-SysID), CARLA_ASU_BRIDGE (https://github.com/ebrahimabdelghfar/Carla_ASU_Bridge) [37], and the accompanying MPC and benchmarking packages, which are released together with the scenario definitions, the exported result CSVs and the figure and sweep generators at https://github.com/ebrahimabdelghfar/Ebrahim_Master_Thesis_repo/tree/feat-follow-premade-racing-track. The benchmarking packages compute one-step and multi-step prediction metrics online (RMSE, MAE, NRMSE, MaxAE, bias, standard deviation and R^2 by Welford's algorithm), compare estimated against ground-truth per-wheel lateral forces, and record controller-switching behaviour, and were built for this study.

Acknowledgment

The On-Track-SysID pipeline studied here originates from the ForzaETH Race Stack and the work of Dikici *et al.* [6], released under an open-source licence; this paper reports a port, a correction, and an extension of that work rather than a reimplementation from scratch.

Funding

This work received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of Interest

The authors declare no conflict of interest.

Ethics

All results in this paper are obtained in simulation. No experiment involved human subjects, personal data, or the operation of a physical vehicle, so no human-subjects approval and no vehicle-safety protocol were required.

Data and Code Availability

The identification package, the ROS 2 bridge, the MPC and the benchmarking packages, the scenario definitions, the exported result CSVs behind every figure and table of Section VI, and the sweep generator behind Table V are in the repositories listed in Appendix B; the top-level project repository is https://github.com/ebrahimabdelghfar/Ebrahim_Master_Thesis_repo/tree/feat-follow-premade-racing-track.

On-Track-SysID is an extension of the ForzaETH release of [6] and inherits its licence; CARLA_ASU_Bridge is GPL-3.0. The two are used as separate ROS 2 overlays communicating only over ROS 2 topics and services, and neither links against the other, so no combined-work obligation arises between them; the identification and control packages in this work are released under the terms the On-Track-SysID upstream requires.

References

[1] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.

[2] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.

[3] N. Baumann, E. Ghignone, J. Kühne, N. Bastuck, J. Becker, N. Imholz, T. Kränzlin, T. Y. Lim, M. Lötscher, L. Schwarzenbach, L. Tognoni, C. Vogt, A. Carron, and M. Magno, "ForzaETH race stack—scaled autonomous head-to-head racing on fully commercial off-the-shelf hardware," *Journal of Field Robotics*, 2024, preprint: arXiv:2403.11784.

[4] H. B. Pacejka, *Tire and Vehicle Dynamics*, 3rd ed. Oxford, UK: Butterworth-Heinemann, 2012.

[5] J. Kabzan, M. I. Valls, V. J. F. Reijgwart, H. F. C. Hendrikx, C. Ehmke, M. Prajapat, A. Bühler, N. Gosala, M. Gupta, R. Sivanesan, A. Dhall, E. Chisari, N. Karnchanachari, S. Brits, M. Dangel, I. Sa, R. Dubé, A. Gawel, M. Pfeiffer, A. Liniger, J. Lygeros, and R. Siegwart, "AMZ driverless: The full autonomous racing system," *Journal of Field Robotics*, vol. 37, no. 7, pp. 1267–1294, 2020.

[6] O. Dikici, E. Ghignone, C. Hu, N. Baumann, L. Xie, A. Carron, M. Magno, and M. Corno, "Learning-based on-track system identification for scaled autonomous racing in under a minute," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, 2025, preprint: arXiv:2411.17508. Code: https://github.com/ForzaETH/On-Track-SysID.

[7] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.

[8] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annual Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 78. PMLR, 2017, pp. 1–16. [Online]. Available: https://proceedings.mlr.press/v78/dosovitskiy17a.html

[9] E. Bakker, L. Nyborg, and H. B. Pacejka, "Tyre modelling for use in vehicle dynamics studies," SAE International, SAE Technical Paper 870421, 1987.

[10] H. B. Pacejka and E. Bakker, "The magic formula tyre model," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, 1992.

[11] F. Farroni, R. Lamberti, N. Mancinelli, and F. Timpone, "TRIP-ID: A tool for a smart and interactive identification of Magic Formula tyre model parameters from experimental data acquired on track or test rig," *Mechanical Systems and Signal Processing*, vol. 102, pp. 1–22, 2018.

[12] T. A. Wenzel, K. J. Burnham, M. V. Blundell, and R. A. Williams, "Dual extended Kalman filter for vehicle state and parameter estimation," *Vehicle System Dynamics*, vol. 44, no. 2, pp. 153–171, 2006.

[13] R. Rajamani, D. Piyabongkarn, J. Y. Lew, K. Yi, and G. Phanomchoeng, "Algorithms for real-time estimation of individual wheel tire-road friction coefficients," *IEEE/ASME Transactions on Mechatronics*, vol. 17, no. 6, pp. 1183–1195, 2012.

[14] C. Ahn, H. Peng, and H. E. Tseng, "Robust estimation of road friction coefficient," *IEEE Transactions on Control Systems Technology*, vol. 21, no. 1, pp. 1–13, 2013.

[15] M. Acosta, S. Kanarachos, and M. Blundell, "Road friction virtual sensing: A review of estimation techniques with emphasis on low excitation approaches," *Applied Sciences*, vol. 7, no. 12, p. 1230, 2017.

[16] Y.-H. J. Hsu, S. M. Laws, and J. C. Gerdes, "Estimation of tire slip angle and friction limits using steering torque," *IEEE Transactions on Control Systems Technology*, vol. 18, no. 4, pp. 896–907, 2010.

[17] L. Hewing, A. Liniger, and M. N. Zeilinger, "Cautious NMPC with Gaussian process dynamics for autonomous miniature race cars," in *Proc. European Control Conference (ECC)*, 2018.

[18] L. Hewing, J. Kabzan, and M. N. Zeilinger, "Cautious model predictive control using Gaussian process regression," *IEEE Transactions on Control Systems Technology*, 2020.

[19] J. Chrosniak, J. Ning, and M. Behl, "Deep dynamics: Vehicle dynamics modeling with a physics-constrained neural network for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5292–5297, 2024, preprint: arXiv:2312.04374.

[20] Z. Wu, C. Hu, Y. Wang, L. Xie, and H. Su, "Vision-augmented on-track system identification for autonomous racing via attention-based priors and iterative neural correction," 2026. [Online]. Available: https://arxiv.org/abs/2603.09399

[21] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, preprint: arXiv:2206.11893.

[22] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. International Conference on Learning Representations (ICLR)*, 2022, preprint: arXiv:2111.00396.

[23] C. Oeltjen, C. Sobolewski, S. Faghfoorian, L. Domokos, G. Vidal, S. Yerramsetty, and I. Ruchkin, "Online slip detection and friction coefficient estimation for autonomous racing," 2025. [Online]. Available: https://arxiv.org/abs/2509.15423

[24] E. Fiala, "Seitenkräfte am rollenden luftreifen," *VDI-Zeitschrift*, vol. 96, no. 29, pp. 973–979, 1954.

[25] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," in *International Conference on Learning Representations (ICLR)*, 2023.

[26] NVIDIA Corporation, "PhysX SDK documentation: Vehicles — tire force computation (PxVehicleComputeTireForceDefault)," Software documentation, 2018, https://nvidia-omniverse.github.io/PhysX/physx/.

[27] M. Brunner, E. Ghignone, N. Baumann, and M. Magno, "R-CARLA: High-fidelity sensor simulations with interchangeable dynamics for autonomous racing," 2025. [Online]. Available: https://arxiv.org/abs/2506.09629

[28] S. Sagmeister, P. Kounatidis, S. Goblirsch, and M. Lienkamp, "Analyzing the impact of simulation fidelity on the evaluation of autonomous driving motion control," 2026. [Online]. Available: https://arxiv.org/abs/2602.07984

[29] J. Abdo, A. Shibu, M. Saeed, A. M. Aga, A. Sivaprazad, and M. Al-Musleh, "Simulating an autonomous system in CARLA using ROS 2," 2025. [Online]. Available: https://arxiv.org/abs/2511.11310

[30] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in *Proc. NeurIPS 2019 Competition and Demonstration Track*, ser. Proceedings of Machine Learning Research, vol. 123. PMLR, 2020, pp. 77–89. [Online]. Available: https://proceedings.mlr.press/v123/o-kelly20a.html

[31] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, 2022, preprint: arXiv:1910.13753.

[32] G. Frison and M. Diehl, "HPIPM: A high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine (Proc. 21st IFAC World Congress)*, vol. 53, no. 2, pp. 6563–6569, 2020.

[33] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.

[34] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[35] R. Bellman and K. J. Åström, "On structural identifiability," *Mathematical Biosciences*, vol. 7, no. 3, pp. 329–339, 1970.

[36] P. Kaur, S. Taghavi, Z. Tian, and W. Shi, "A survey on simulators for testing self-driving cars," in *Proc. 4th Int. Conf. on Connected and Autonomous Driving (MetroCAD)*, 2021, pp. 62–70. [Online]. Available: https://arxiv.org/abs/2101.05337

[37] E. Abdelghfar, "Formula AI simulator: A CARLA-based simulation bridge for autonomous formula student vehicles," Software, GPL-3.0, 2026, https://github.com/ebrahimabdelghfar/Carla_ASU_Bridge.

[38] T. Foote and M. Purvis, "Standard units of measure and coordinate conventions," Open Source Robotics Foundation, ROS Enhancement Proposal 103, 2010, https://www.ros.org/reps/rep-0103.html.

[39] SAE International, "Vehicle dynamics terminology," SAE International, Surface Vehicle Recommended Practice J670, 2008.

[40] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[41] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[42] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[43] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[44] J. A. Nelder and R. Mead, "A simplex method for function minimization," *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.

[45] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[46] M. A. Branch, T. F. Coleman, and Y. Li, "A subspace, interior, and conjugate gradient method for large-scale bound-constrained minimization problems," *SIAM Journal on Scientific Computing*, vol. 21, no. 1, pp. 1–23, 1999.

[47] P. Virtanen, R. Gommers, T. E. Oliphant, *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[48] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.

[49] J. Svendenius and M. Gåfvert, "A brush-model-based semi-empirical tire-model for combined slips," SAE International, SAE Technical Paper 2004-01-1064, 2004.

[50] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[51] Institute of Automotive Technology, Technical University of Munich, "global_racetrajectory_optimization," Software, Institute of Automotive Technology, Technical University of Munich, 2019–2022, https://github.com/TUMFTM/global_racetrajectory_optimization.

[52] G. C. Goodwin and R. L. Payne, *Dynamic System Identification: Experiment Design and Data Analysis*, ser. Mathematics in Science and Engineering. New York, NY, USA: Academic Press, 1977, vol. 136.